

Agentic 時代 軟體開發準則

從單一專案到跨專案協作的規格體系、撰寫規範與實戰範例

版本	1.1.0
狀態	ready
日期	2026-10-07
適用技術	Dart / Flutter、Java、C#
主要讀者	Agent、工程師、PM、設計師、技術主管
對應 Markdown	agentic-development-guideline.md

目錄

第 0 章 文件定位與使用方式

- 0.1 目的
- 0.2 適用範圍與讀者
- 0.3 如何使用本文件
- 0.4 規範用語
- 0.5 本文件的版本與修改

第 1 章 核心理念與十二原則

- 1.1 讀者變了
- 1.2 規格硬度階梯
- 1.3 十二原則
- 1.4 Human in the Loop (HITL) 標記約定

第 2 章 規格體系總覽

- 2.1 分層模型
- 2.2 單一專案的目錄結構
- 2.3 文件清單矩陣
- 2.4 規格粒度階梯
- 2.5 依規模決定最小文件集
- 2.6 文件狀態生命週期

第 3 章 通用文件格式

- 3.1 Front-matter 標準欄位
- 3.2 ID 命名規則
- 3.3 參照語法
- 3.4 需求撰寫句型
- 3.5 場景格式 (Given / When / Then)
- 3.6 驗證標記
- 3.7 單一事實來源對照表

第 4 章 各規格書規範

- 4.1 AGENTS.md (L0：agent 操作手冊)
- 4.2 STATUS.md (執行層：交接記憶)
- 4.3 Product Intent (L1：意圖規格)
- 4.4 Glossary & Invariants (L2：術語與不變量)
- 4.5 Domain Model (L2：領域模型)
- 4.6 Architecture Spec (L3：架構規格)
- 4.7 ADR (L3：決策紀錄)
- 4.8 Pattern Policy (L3：模式政策)
- 4.9 Feature Spec (L4：功能規格，最核心)
- 4.10 State Machine Spec (L4：狀態機)
- 4.11 Flow / Sequence Spec (L4：流程)
- 4.12 Contract Spec (L5：契約)
- 4.13 UX Spec (L6：UI/UX 規格)
- 4.14 Quality Spec (L7：驗收與評測)
- 4.15 Autonomy Policy (L8：自主權限)
- 4.16 Ops Runbook (L9：營運)
- 4.17 Agent Spec 與 Tool Spec (L10：產品內含 agent)
- 4.18 Task Spec (執行層：任務單)
- 4.19 Change Request (執行層：變更請求)
- 4.20 Epic (組織層：跨功能與跨專案計畫)

第 5 章 UML 與 GoF 設計模式的定位

- 5.1 角色改變：從「描述設計」到「約束設計」
- 5.2 保留的 6 種 UML 圖
- 5.3 架構測試範例 (把圖變成可執行的規則)
- 5.4 GoF → Pattern Policy
- 5.5 產品含 agent 時：agent 設計模式

第 6 章 追溯、交叉參照與變更影響

- 6.1 追溯鏈
- 6.2 雙向追溯
- 6.3 變更影響分析 (Change Impact)
- 6.4 版本與相容性
- 6.5 參照衛生規則
- 6.6 功能地圖 `_index.md`

第 7 章 協作規範

- 7.1 同一專案內：多功能、多 agent 並行
- 7.2 跨專案協作

第 8 章 Agent 執行協議

- 8.1 載入順序 (Context Loading)
- 8.2 開工前檢查 (Preflight)
- 8.3 執行中規則
- 8.4 不確定時的決策樹
- 8.5 升級條件 (需停止並回報人工)
- 8.6 完工與交接
- 8.7 PR 描述模板

第 9 章 品質閘門與自動化

- 9.1 CI 閘門
- 9.2 `spec_lint`：規格檢查工具 (參考實作)
- 9.3 追溯檢查 (`covers` 標記)
- 9.4 Eval 閘門 (含 LLM/agent 的產品)

第 10 章 實戰範例 (讀者是 Agent)

- 10.1 單一專案：建立 ShopLite 的規格基礎
- 10.2 單一功能：F-012「購物車修改數量」
- 10.3 多功能並行：三個 agent 同時開工
- 10.4 多功能互相參照：F-020「結帳」與變更影響
- 10.5 多專案協作：App + API + 共用契約庫

第 11 章 導入、治理與持續改進

- 11.1 角色與責任
- 11.2 規格變更流程
- 11.3 規格健康度指標
- 11.4 失敗回饋迴路 (P12)
- 11.5 導入路線

附錄 A 檢查表

- A.1 Feature Spec 審查檢查表
- A.2 Task 審查檢查表
- A.3 PR 審查檢查表 (人)
- A.4 新專案啟動檢查表

附錄 B 反模式清單

附錄 C 問題 → 去哪裡找

附錄 D Human in the Loop (HITL) 總覽索引

- D.1 事前核准 (HITL・事前核准)
- D.2 事中澄清 (HITL・事中澄清)
- D.3 事後審查 (HITL・事後審查)
- D.4 例外升級 (HITL・例外升級)
- D.5 常見誤用
- D.6 已知缺口 (建議下版補強)

第 0 章 文件定位與使用方式

0.1 目的

本文件是 agentic 開發團隊的**共同開發準則**，規範三件事：

- 1. **該有哪些規格書**：從一個專案到一個功能，每一層需要什麼文件。
- 2. **每份規格書怎麼寫**：目的、必要章節、格式、反模式、驗證方式。
- 3. **多人、多 agent、多專案如何協作**：如何分工、參照、變更、交接。

核心主張只有一句：

規格書不是敘述文，而是「可驗證的契約」。能變成測試、型別、schema、編譯器檢查的，就不要只寫成文字。

0.2 適用範圍與讀者

讀者	閱讀目的	優先閱讀
Agent (AI 程式代理)	在沒有記憶的情況下，每次 session 都能依規範正確工作	第 1、3、8 章，以及任務指向的規格
工程師	撰寫、審查規格，設定 CI 與治理	第 2-7、9 章
PM / 設計師	撰寫意圖、UX 規格，回答開放問題	第 3、4.3、4.13 節
技術主管	導入、治理、衡量成效	第 1、11 章

適用規模：單人專案、單一團隊多功能、多 agent 平行作業、多 repo 跨專案協作。

0.3 如何使用本文件

你要做的事	看哪裡
開一個新專案	2.2 (目錄) → 4.1、4.2 → 範例 10.1
寫一個新功能的規格	4.9 → 範例 10.2
同時開多個功能給多個 agent	7.1 → 範例 10.3
功能之間有依賴或互相引用	第 6 章 → 範例 10.4
前端、後端、共用契約分屬不同 repo	7.2 → 範例 10.5
我是 agent，剛被指派任務	第 8 章 (執行協議)
審查規格或 PR	附錄 A
想知道哪些節點需要人介入	1.4、附錄 D

0.4 規範用語

用語	意義
MUST / 必須	絕對要求，違反即為缺陷，CI 應能偵測
MUST NOT / 禁止	絕對禁止
SHOULD / 應該	強烈建議，偏離時須在文件或 PR 中說明理由
MAY / 可以	選用

規格中的每條需求**必須**標示強度。本文件自身亦遵守此規則。

0.5 本文件的版本與修改

- 本文件存放於 repo：[docs/00-guideline/agentic-guideline.md](#)，與程式碼同版本控管。
- **【HITL・事前核准】** 修改須經 PR 與至少一位技術負責人核准，並依語意化版本遞增（破壞性規範變更升主版號）。

- **失敗回饋迴路**：每次 agent 因規格不足而出錯，必須回頭修正規格（見 11.4）。本文件會因此持續演進。

修訂紀錄

版本	日期	變更
1.0.0	2026-10-07	初版
1.1.0	2026-10-07	新增 1.4「HITL 標記約定」；於既有規則標註 HITL ；新增附錄 D「HITL 總覽索引」（相容變更，未改動既有規則內容）

第 1 章 核心理念與十二原則

1.1 讀者變了

過去規格書的讀者是人，人會腦補、會開會追問、會記得上週的決定。現在主要的讀者是 agent：

agent 的特性	對規格書的要求
每次 session 沒有記憶	所有決策 必須 寫進 repo，不得只存在對話或人腦
context 有限	文件小而分層，可按需載入，單份建議 ≤ 300 行
遇到空白會用「看似合理」的猜測填補	明確寫出邊界、非目標、已決定事項、不確定時的行為
能執行指令、讀測試結果	每條需求配 可執行的驗證方式
字面理解	禁用模糊詞，用數字與範例
可平行工作	任務有明確的檔案範圍與介面契約，避免互相衝突
容易過度設計或結構漂移	用模式政策、分層規則與架構測試約束

1.2 規格硬度階梯

同一條規則，可以用不同「硬度」表達。**越往下越可靠，應盡量往下推。**

硬度	形式	例子	違反時
1（軟）	散文描述	「購物車數量不要太大」	沒人發現
2	精確的條列需求	「R1 (MUST)：數量 1..99」	code review 才發現
3	範例與場景（Given/When/Then）	「數量 99 時按 + 不送請求」	人工比對
4	可執行測試	<code>quantity_test.dart</code>	CI 失敗
5（硬）	型別 / schema / 編譯器	<code>sealed class</code> 、OpenAPI schema、架構測試	無法編譯、無法合併

準則：每條 MUST 級需求，至少要到達硬度 4。能到 5 的優先到 5。

1.3 十二原則

編號	原則	說明
P1	規格即契約	每條需求必須可驗證；無法驗證的需求不得標為 MUST
P2	單一事實來源（SSOT）	同一資訊只在一處定義，其他處以參照連結，不複製
P3	寫進 repo	規格與程式碼同版本、同 PR 更新，變更可追溯
P4	數字取代形容詞	「300 ms」而非「很快」；「≥ 4.5:1」而非「清楚」
P5	明寫非目標與已決定事項	防止 agent 過度發揮或推翻既有決策
P6	不確定時停下並記錄	不得猜測；規格缺漏須記入待釐清，等待人工回答。【HITL・事中澄清】
P7	結構規則機器化	分層、狀態轉換、模式使用，盡量轉為測試或編譯期檢查
P8	範例優先於說明	每個規範配「專案內真實檔案」作為模仿對象，並附反例
P9	小而分層	一份文件一個目的；agent 按需載入，不需通讀全部
P10	權限明確	明定 agent 可自主、需核准、禁止的動作與升級條件。【HITL・事前核准】 【HITL・例外升級】
P11	可追溯	需求 → 設計 → 程式 → 測試，雙向可查；變更可分析影響
P12	失敗驅動演進	agent 每次出錯，先問「哪份規格少寫了什麼」並補回

1.4 Human in the Loop (HITL) 標記約定

本準則最重要的安全機制之一，是在關鍵節點**安排人類介入**，而不是讓 agent 從頭到尾自行完成。這些機制散布在各章，為了一眼可見，凡屬 HITL 的規則，文件中一律以 **【HITL・類型】** 標記，並彙整於**附錄 D**。

標記	類型	定義	Agent 在此節點的行為
【HITL・事前核准】	執行前須經人核准	未獲核准不得進行	提出計畫／CR／PR，然後 等待
【HITL・事中澄清】	不確定時向人提問	規格缺漏、矛盾、或需要人才能決定的事	停止相關工作，記入「待釐清」，等人回答
【HITL・事後審查】	產出後由人驗收	不通過則退回	附上完整證據（PR 模板、測試結果）
【HITL・例外升級】	觸發門檻或超出能力時轉交人	達到升級條件	停止，回報現象與已嘗試的做法

設計原則

- 依風險決定介入程度**：低風險（A 級）由 agent 自主、事後審查（人在迴路旁，*on the loop*）；高風險（B／C 級、**draft → ready**、契約變更）必須事前核准（人在迴路中，*in the loop*）。
- 核准權永遠在人**：agent 不得核准自己提出的規格、CR 或 PR。
- 介入點必須寫進文件**，不得只靠口頭約定；**核准與回答必須留下可追溯的紀錄**（PR 核准、CR 決議、STATUS.md 的回覆）。
- 避免人變成瓶頸或橡皮圖章**：介入太多，人會疲勞而隨手同意；太少則風險失控。以「待釐清平均等待時間」與「停止並回報次數」衡量（見 11.3）。

第 2 章 規格體系總覽

2.1 分層模型

組織層 (多專案)	本準則 · 共用契約庫(platform-contracts) · Epic	
專案層	L0 AGENTS.md	agent 操作手冊 (每次必讀)
	L1 Product Intent	為什麼做、不做什麼
	L2 Domain	術語、不變量、領域模型
	L3 Architecture	架構、依賴規則、ADR、模式政策
功能層	L4 Feature Spec	行為需求、場景、狀態機、流程
	L5 Contract	API / Schema / ERD (機器可讀)
	L6 UX Spec	Design Token、元件狀態矩陣
品質層	L7 Quality	驗收、eval、架構測試
治理層	L8 Autonomy Policy	agent 權限與升級條件
營運層	L9 Ops	Runbook、可觀測性
產品層	L10 Agent / Tool Spec	(僅當產品本身含 agent)
執行層	Task · STATUS · CR	任務單、交接記憶、跨專案變更請求

閱讀方向：由上而下是「約束越來越具體」；agent 執行任務時由 L0 開始，依任務單指向的文件向下載入，不需通讀全部。

2.2 單一專案的目錄結構

repo/		
├ AGENTS.md	# L0	
├ STATUS.md	# 交接記憶	
├ docs/		
│ └ 00-guideline/		
│ │ └ agentic-guideline.md	# 本準則	
│ └ 01-intent/		
│ │ └ PI-001-shopping-cart.md		
│ └ 02-domain/		
│ │ └ glossary-invariants.md		
│ │ └ domain-model.md	# 類別圖 + 聚合規則	
│ └ 03-architecture/		
│ │ └ architecture.md	# C4 / 元件圖 + 依賴規則	
│ │ └ pattern-policy.md		
│ │ └ adr/ADR-0004-cart-source-of-truth.md		
│ └ 04-features/		
│ │ └ _index.md	# 功能地圖：依賴、擁有者、狀態	
│ │ └ F-012-cart-quantity.md		
│ │ └ SM-order.md	# 狀態機	
│ │ └ flows/FLOW-001-checkout.md		
│ └ 05-contracts/		
│ │ └ openapi.yaml		
│ │ └ schemas/*.json		
│ │ └ erd.md		
│ └ 06-ux/		
│ │ └ design-tokens.json		
│ │ └ components/UX-quantity-stepper.md		
│ └ 07-quality/		
│ │ └ acceptance/		
│ │ └ evals/		
│ └ 08-policy/		
│ │ └ autonomy-policy.md		
│ └ 09-ops/		
│ │ └ runbook.md		
│ └ 10-agents/	# 僅產品含 agent 時	
│ │ └ AG-support.md		
│ │ └ TOOL-create-return.yaml		
│ └ epics/		
│ │ └ E-2026-014-stock-aware-cart.md		
├ tasks/		
│ └ T-0231.md		
├ tool/		
│ └ spec_lint.dart	# 規格檢查 (見第 9 章)	
├ test/		
│ └ architecture/	# 架構規則測試	

2.3 文件清單矩陣

ID 前綴	文件	層	目的 (一句話)	主要讀者	驗證方式
—	AGENTS.md	L0	讓 agent 在零記憶下知道指令、規則、禁區	agent	指令可執行；CI 對照

—	STATUS.md	執行	跨 session 的外部記憶與交接	agent、人	與 PR 狀態一致
PI	Product Intent	L1	說明為何做、成功指標、 非目標	PM、agent	指標可量測
INV / TERM	Glossary & Invariants	L2	消除語意歧義，定義永遠成立的規則	全部	不變量對應測試
DOM	Domain Model	L2	實體、關係、聚合邊界	agent	類別圖 + 圖外約束
ARCH	Architecture	L3	模組邊界、依賴方向、非功能指標	agent、工程師	架構測試
ADR	Decision Record	L3	記錄決策與否決方案，防止被推翻	全部	狀態與重審條件
PP	Pattern Policy	L3	規定哪裡用哪個模式、哪些禁止	agent	code review + lint
F	Feature Spec	L4	功能的行為需求與場景 (核心)	agent、QA	場景 → 測試
SM	State Machine	L4	有狀態實體的合法轉換	agent	轉換表驅動窮舉測試
FLOW	Flow / Sequence	L4	跨模組、跨服務的呼叫順序與失敗路徑	agent	整合測試
C	Contract	L5	API / schema 的機器可讀契約	前後端 agent	契約測試
UX	UX Spec	L6	Token、元件狀態矩陣、無障礙	設計、agent	Golden test
ACC / EVAL	Acceptance / Eval	L7	驗收條件與 LLM 評測集	QA、agent	CI 門檻
AP	Autonomy Policy	L8	agent 權限等級與升級條件	全部	審計紀錄
OPS	Runbook	L9	部署、監控、告警與處置	維運、agent	演練
AG / TOOL	Agent / Tool Spec	L10	產品內 agent 的職責、護欄、工具	agent 開發	eval
T	Task Spec	執行	一次性工單，限定範圍與完成條件	agent	完成條件勾選
CR	Change Request	執行	跨專案、跨擁有者的變更申請	各專案 agent	對方 repo 接受紀錄
E	Epic	組織	跨功能、跨專案的整體計畫與發布順序	全部	子項目狀態彙整

2.4 規格粒度階梯

組織 → 專案 → 領域/模組 → 功能(F) → 需求(R) → 場景(S) → 測試 → 任務(T)
 本準則 AGENTS DOM/ARCH F-012 F-012#R3 F-012#S3 *_test T-0231
 Epic PI INV/ADR

每一層都有穩定的 ID，可被下一層參照（見 3.2、3.3）。

2.5 依規模決定最小文件集

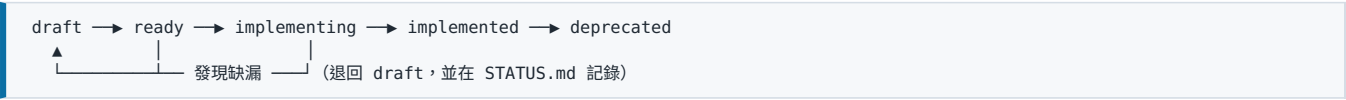
並非每個變更都要寫齊所有文件。依規模選擇「最小可行規格」：

規模	例子	必要文件	視情況追加
S （修正）	修 bug、改文案	Task（含重現步驟與驗證）	—
M （單一功能）	購物車調整數量	Task + Feature Spec	有狀態 → SM；有畫面 → UX；有 API → C
L （跨功能 / 新模組）	庫存感知購物車	Epic + 多份 Feature + Contract + ADR	FLOW、Pattern Policy 補充
XL （跨專案）	App + API + 共用契約	Epic（置於契約庫）+ CR + Contract +	發布順序、相容性計畫

		各 repo 的 Feature / Task	
新專案	從零開始	L0-L3 全套 + STATUS	依產品性質追加 L5-L10

規則 G-2.5：規格的投入必須與風險成正比。以下情況一律升級到至少 M 級規格：涉及金流、權限、個資、資料遷移、不可逆操作。

2.6 文件狀態生命週期



狀態	意義	Agent 可做的事
draft	撰寫中，可能含開放問題	不得實作；僅可協助補寫或提問
ready	已審查、無阻擋性開放問題、已定義驗證方式	可實作
implementing	有任務進行中	可繼續；變更須通知任務擁有者
implemented	已實作且驗證通過	修改須走變更流程（第 6 章）
deprecated	已廢止，保留供追溯	不得依賴；須標明替代文件

規則 G-2.6a：agent 只實作 **ready** 或 **implementing** 的規格。遇到 **draft** 一律停止並回報。【HITL・例外升級】規則 G-2.6b：**ready** 的 Feature 不得依賴 **draft** 的文件（由 `spec_lint` 檢查）。規則 G-2.6c：**draft** → **ready** 的狀態變更必須由人核准（規格擁有者）；agent 只能撰寫草稿與提出問題，不得自行將規格改為 **ready**。【HITL・事前核准】

第 3 章 通用文件格式

所有規格文件（除 AGENTS.md、STATUS.md 外的 docs/**/*.*md）必須遵守本章。

3.1 Front-matter 標準欄位

每份文件開頭必須有 YAML front-matter，供人與工具解析：

```
---
id: F-012                # 必填，專案內唯一
type: feature            # 必填：intent|domain|arch|adr|policy|feature|state|flow|
                        #      contract|ux|quality|ops|agent|tool|task|cr|epic
title: 購物車：修改數量 # 必填
status: ready            # 必填：draft|ready|implementing|implemented|deprecated
owner: team-cart         # 必填：負責人或團隊（人工，非 agent）
version: 1.2.0           # 建議：語意化版本
depends_on: [F-010, C-CART-API@1.3] # 依賴：不滿足則無法實作
refs: [INV-001, INV-003, ADR-0004, UX-quantity-stepper] # 參考：知道即可
verify: [test/features/cart/quantity_test.dart]        # 驗證：測試路徑或指令
supersedes: []    # 取代哪些舊文件
updated: 2026-10-07
---
```

欄位語意區分：- depends_on：被依賴者變更可能使本文件失效。須做影響分析。- refs：僅為背景參考，被參照者變更時提示檢視即可。

3.2 ID 命名規則

對象	格式	例
功能	F-nnn	F-012
需求（功能內）	R<n>	R3
場景（功能內）	S<n>	S3
不變量	INV-nnn	INV-001
架構規則	ARCH-nnn	ARCH-001
決策紀錄	ADR-nnnn	ADR-0004
模式政策	PP-nnn	PP-003
狀態機	SM-<名稱>	SM-order
流程	FLOW-nnn	FLOW-001
契約	C-<名稱>	C-CART-API
UX 元件	UX-<名稱>	UX-quantity-stepper
任務	T-nnnn	T-0231
變更請求	CR-nnnn	CR-0007
Epic	E-yyyy-nnn	E-2026-014
測試覆蓋標記	covers: <ID>	// covers F-012#R3

規則：- ID 一經發布不得重用；文件廢止後 ID 保留。- ID 不得帶語意會變動的內容（例如不要用 F-cart-v2）。- 跨專案時加專案前綴：<project>:<ID>，如 shop-api:F-107。

3.3 參照語法

目的	語法	例
參照整份文件	[[ID]]	[[F-010]]
參照需求或場景	[[ID#Rn]] / [[ID#Sn]]	[[F-012#R3]]
指定最低版本	[[ID@x.y]]	[[C-CART-API@1.3]]
跨專案	[[project:ID]]	[[shop-api:F-107#R2]]

規則：- MUST NOT 複製被參照處的內容，只寫參照。複製會造成兩份事實，日後必然不一致（違反 P2）。- 參照契約時必須帶版本

(@x.y)，否則視為依賴最新版而無法判斷相容性。 - `depends_on` / `refs` 與內文 `[[...]]` 應一致；`spec_lint` 會檢查指向是否存在。

3.4 需求撰寫句型

一條需求 = 一個主體 + 一個強度詞 + 一個可驗證的行為 + 具體數值（若適用）。

R2 (MUST)：當使用者變更數量後，系統必須在 300 ms 內送出請求 (debounce)，請求進行中，+/- 按鈕必須為 disabled。

禁用詞與替代寫法

禁用詞	問題	替代寫法
快速、即時	無法驗證	「 $P95 \leq 800\text{ ms}$ 」
友善、直覺	主觀	「首次使用者在 30 秒內完成加入購物車（可用性測試）」
適當、合理	留給 agent 猜	列舉具體條件與表格
盡量、儘可能	無強度	改為 SHOULD 並寫出偏離需附理由
等等、其他	開放集合	列舉完整清單，或寫「僅限下列」
類似 X	邊界不明	指向 X 的檔案路徑並寫出差異
必要時	無觸發條件	寫出觸發條件
支援多種	數量不明	列出支援清單

3.5 場景格式 (Given / When / Then)

```
### S2: 超過上限 (驗證 R1)
Given SKU-A 數量為 99
When 使用者點擊「+」
Then 數量維持 99；「+」呈 disabled；不發送任何請求
```

- 每個場景必須標明驗證哪條需求（驗證 R1）。
- 每條 MUST 需求至少有一個場景；每個場景至少對應一個測試。
- 場景使用具體資料 (SKU-A、99)，不使用「某個商品」。

3.6 驗證標記

規格 → 測試，測試 → 規格，雙向標記：

```
<!-- 規格內 -->
驗證：`test/features/cart/quantity_test.dart` (S1-S3)
```

```
// 測試內
test('S2 超過上限不送請求', () {
  // covers F-012#R1, F-012#S2
  ...
});
```

3.7 單一事實來源對照表

資訊	唯一所在處	其他文件的做法
術語定義	<code>glossary-invariants.md</code>	參照 <code>TERM-xxx</code>
永遠成立的規則	<code>glossary-invariants.md</code> (INV)	參照 <code>INV-xxx</code> ，測試標 <code>covers</code>
API 形狀與錯誤碼	<code>openapi.yaml</code> / <code>schema</code>	功能規格只寫「呼叫哪個 operationId」
顏色、間距、字體	<code>design-tokens.json</code>	元件規格參照 token 名稱，不寫色碼
狀態轉換	<code>SM-*.md</code> 的轉換表	功能規格參照 <code>SM-order</code> ，不重述
決策與理由	<code>ADR-*</code>	其他文件寫「見 ADR-0004」
指令（建置、測試）	<code>AGENTS.md</code>	任務單不重複寫指令
目前進度	<code>STATUS.md</code>	規格書內不寫進度

第 4 章 各規格書規範

每一節的固定結構：目的 → 必要章節 → 寫法規則 → 反模式 → 範本。完整填寫後的範例見第 10 章。

4.1 AGENTS.md (L0：agent 操作手冊)

目的：相當於 agent 的「入職手冊」。每個 session 開頭必讀，所以必須短，只放「不寫就一定會做錯」的內容。

必要章節

章節	內容
專案一句話	做什麼、技術棧、版本
指令	安裝、分析、測試、格式化、產生程式碼——可直接複製執行
完成的定義 (DoD)	一個任務算完成必須滿足的檢查
架構規則 (MUST)	分層、狀態管理、國際化等硬規則
禁區 (NEVER)	不得修改的路徑與不得做的事
不確定時	遇到缺漏或矛盾該怎麼辦
跨專案規則	(多專案時) 不得修改他人 repo，如何提 CR
文件導覽	各類資訊在哪裡

寫法規則 - **MUST** ≤ 100 行。超過就把細節移到 `docs/` 並以連結導覽。 - 只寫指令與規則，不寫願景、背景、故事。 - 每條規則都必須能「判斷有沒有違反」。 - **MUST** 包含「不確定時怎麼辦」，這是降低亂猜最有效的一段。 **[HITL・事中澄清]**

反模式：把整份架構文件貼進去；寫「請寫出高品質程式碼」；指令過期而無人更新。

範本：見 10.1。

4.2 STATUS.md (執行層：交接記憶)

目的：agent 沒有跨 session 記憶，`STATUS.md` 就是它的外部大腦，也是人類了解進度的唯一入口。

必要章節：進行中・已完成・待釐清(需人工回答)・已知問題與技術債・下一步建議・最後更新者與時間。

寫法規則 - 每次 session 結束前 **MUST** 更新 (列入 DoD)。 - 「待釐清」每項要有編號 (Q-nn)、影響範圍、提問時間；人工回答後移到對應規格並從此處刪除。 **[HITL・事中澄清]** - 只記錄狀態與問題，不記錄決策理由 (理由進 ADR)。 - 保持 ≤ 100 行；已完成項目定期歸檔到 `docs/changelog.md`。

反模式：把它當日誌寫成流水帳；問題寫了卻沒人回答也沒人追。

4.3 Product Intent (L1：意圖規格)

目的：說明為什麼做、為誰做、怎樣算成功、明確不做什麼。agent 最常犯的錯是做過頭，所以「非目標」是最有價值的段落。

必要章節

章節	說明
目標使用者	具體描述情境 (手機、單手、網路不穩)
要解決的問題	一兩句話
成功指標	可量測的數字與量測方式
非目標	明確列出不做的事
已決定事項	不得重新討論的決策
開放問題	每項標示負責人與期限

寫法規則 - 指標必須有數字與基準 (例如「完成率 ≥ 65%」)。 - 開放問題有負責人與期限，agent 看到就知道此處不得自行決定。 **[HITL・事中澄清]** - 與功能規格的關係：Feature 的 `refs` 必須指向其 PI。

反模式：只有願景沒有非目標；指標寫「提升使用者滿意度」。

4.4 Glossary & Invariants (L2：術語與不變量)

目的：消除語意歧義；不變量是「永遠成立的規則」，可直接轉成測試。

必要章節 - 術語表：術語・定義・不是什麼（排除常見誤解）。 - 不變量：INV-*nnn* 編號、單句可驗證、標明擁有的聚合。

寫法規則 - 一個概念只有一個名字；程式碼中的類別名稱必須與術語一致。 - 不變量一律用「必須 / 不得」+ 可判定條件，例如「*CartItem.quantity* 為 1..99 的整數」。 - 每條 INV 必須至少有一個測試標記 *covers INV-xxx*。

反模式：同一概念出現「訂單 / 單子 / Order」三種叫法；不變量寫成「數量要合理」。

4.5 Domain Model（L2：領域模型）

目的：用類別圖固定實體、關係、多重性與聚合邊界，避免 agent 自行發明資料結構。

必要章節：類別圖（Mermaid *classDiagram*）・圖外約束（聚合根、不可變物件、禁止事項）・與 ERD 的對應。

寫法規則 - 圖只表達結構；agent 不可從圖推論的規則必須寫在圖下方（例如「外部只能透過 Cart 修改 CartItem」）。 - 圖中沒有的 domain 類別，不得新增，須先更新本文件。 - 金額一律值物件 *Money*（整數最小單位），禁止 *double*。

4.6 Architecture Spec（L3：架構規格）

目的：定義模組邊界、依賴方向、非功能指標，並以架構測試強制執行。

必要章節

章節	內容
系統脈絡圖（C4 L1）	系統與外部角色、外部系統
元件圖（C4 L2/L3）	模組、公開介面、依賴方向
依賴規則 ARCH- <i>nnn</i>	例：ARCH-001 domain 不得 import data / presentation
公開介面與禁區	每個模組哪些可被外部呼叫、哪些是內部
非功能需求	效能、安全、可用性的具體數值

寫法規則 - 每條 ARCH 規則必須對應一個架構測試（見 5.3）。 - 非功能需求用數字：「冷啟動 P95 ≤ 2.0 s（Pixel 6）」。 - 圖用 Mermaid（文字化、可 diff），不用圖片檔。

4.7 ADR（L3：決策紀錄）

目的：記錄「為什麼這樣決定」與「否決了什麼」，防止 agent 在幾個月後把決策推翻重做。

必要章節：狀態（Proposed / Accepted / Superseded）・背景・決定・被否決方案與理由・後果・重新檢視條件。

寫法規則 - 決策不可修改，只能被新的 ADR 取代（舊的標 *Superseded by ADR-xxxx*）。 - 「重新檢視條件」要具體（例如「若需要離線編輯，重開此決策」），讓 agent 知道何時可以提出質疑。【HITL・事中澄清】 - 一份 ADR 一個決策，≤ 1 頁。

4.8 Pattern Policy（L3：模式政策）

目的：規定在本專案中哪裡用哪個設計模式、哪些禁止。不是教學，是約束。完整原則見第 5 章。

必要章節：核准清單（情境 → 模式 → 專案內範例路徑）・禁止或限制清單（含理由）・決策規則。

寫法規則 - 每個核准模式必須附一個專案內真實檔案路徑作為範例。 - 決策規則至少包含：①已有範例就沿用；②只有一個實作不建立抽象；③清單外模式需 PR 說明並核准。【HITL・事前核准】

4.9 Feature Spec（L4：功能規格，最核心）

目的：定義單一功能的行為。這是 agent 最常讀、最影響產出品質的文件。

必要章節（順序固定，利於 agent 解析）

#	章節	說明
1	Front-matter	見 3.1，含 <i>depends_on</i> 、 <i>verify</i>
2	目的與範圍	一段話 + 對應的 PI
3	需求	<i>R1..Rn</i> ，每條含強度詞（MUST / SHOULD / MAY）

4	場景	S1...Sn，Given / When / Then，標明驗證哪條需求
5	邊界與錯誤	表格：情況 → 預期行為
6	非功能	效能、無障礙、安全，皆需數字
7	介面與資料	只寫參照：operationId、SM、UX 元件
8	超出範圍	明確不做事
9	驗證	測試檔案路徑與指令
10	開放問題	負責人與期限；有阻擋性問題則狀態不得為 ready。 【HITL・事中澄清】

寫法規則 1. 每條 MUST 需求至少一個場景；每個場景標明驗證對象。2. 「邊界與錯誤」**必須用表格**，agent 不會漏讀。3. 必須有「超出範圍」，防止過度實作。4. 狀態類行為**參照 SM**，不在此重述轉換。5. 單份 ≤ 300 行；過長即拆成多個 Feature。

反模式：需求與設計混寫（把「用 Riverpod 實作」寫進需求）；沒有錯誤路徑；用「類似購物車」帶過。

4.10 State Machine Spec (L4：狀態機)

目的：有生命週期的實體（訂單、付款、購物車）其合法轉換的**唯一定義**。投資報酬率最高的 UML 圖。

必要章節：Mermaid `stateDiagram` · 轉換表（單一事實來源，圖僅作示意） · 終態與禁止規則 · 稽核要求 · 窮舉測試。

寫法規則 - 轉換表欄位：從 · 事件 · 到 · 守衛條件 · 副作用。 - 表格以外的轉換一律禁止，並**必須**回傳指定錯誤（如 `ERR_INVALID_TRANSITION`）。 - 測試**必須**對「所有狀態 × 所有事件」窮舉（N 個狀態、M 個事件 → N×M 個案例）。 - 圖與表不一致時，**以表為準**。

4.11 Flow / Sequence Spec (L4：流程)

目的：描述**跨模組或跨服務**的呼叫順序與失敗路徑。僅在「涉及 ≥ 3 個參與者，或有補償／回復邏輯」時才寫。

必要章節：序列圖（Mermaid `sequenceDiagram`） · **失敗路徑表**（每一步失敗時的補償） · 冪等性與逾時設定 · 驗證（整合測試路徑）。

寫法規則：圖中**每個失敗分支**都要對應表格中的一列；寫出逾時秒數、重試次數、是否冪等。

4.12 Contract Spec (L5：契約)

目的：以機器可讀格式定義介面，agent 可據以產生 client、mock、契約測試。

格式：OpenAPI 3.x · JSON Schema · Protobuf · GraphQL schema · ERD（Mermaid `erDiagram`）。

寫法規則 - **不得**用散文描述 API；以 OpenAPI 為準，Feature 只寫 `operationId`。 - 每個錯誤碼都**必須**寫出並附 schema；欄位**必須**標範圍、格式、必填。 - 變更遵守語意化版本（見 6.4）；破壞性變更**必須**走 CR。【HITL・事前核准】 - CI **必須**包含契約測試（如 Schemathesis、Pact）或 schema 驗證。

4.13 UX Spec (L6：UI/UX 規格)

目的：把視覺與互動轉為 agent 可執行、可驗證的規格。

組成

件	內容	驗證
Design Tokens	顏色、間距、圓角、字體（JSON，單一事實來源）	腳本產生 <code>ThemeData</code>
元件規格	Props、 狀態矩陣 、尺寸、無障礙、動畫	Golden test、Widget test
頁面規格	版面、各狀態（loading / empty / error / offline）、導覽	截圖測試

寫法規則 - **狀態矩陣必須窮舉**：default、hover/pressed、focus、disabled、loading、empty、error。agent 最常漏的就是這些。 - 數值用 token 名稱（`spacing.md`），不用色碼與像素。 - 無障礙為 MUST：觸控目標 ≥ 48×48 dp、對比 ≥ WCAG AA 4.5:1、提供 `Semantics` 標籤。 - 每個狀態對應一張 Golden 圖。

4.14 Quality Spec (L7：驗收與評測)

目的：「測試就是規格的最終形態」。定義驗收門檻，產品含 LLM 時另需 eval 集。

必要內容 - 驗收條件：功能層以場景對應測試；系統層以非功能指標對應壓測／量測。 - **Eval 集**（含 LLM / agent 時）：正向、邊界、**對抗性**（prompt injection）案例；通過門檻；納入 CI。

寫法規則：每個 eval 案例要有 `id`、`input`、`expect`（必呼叫／禁呼叫的工具、必含／禁含的內容）；門檻（如 `pass_threshold: 0.90`）寫在 suite 層級。

4.15 Autonomy Policy（L8：自主權限）

目的：定義 agent「可以自己做什麼、什麼要人核准、什麼禁止、何時必須停下」。本節是整份準則中 HITL 的總開關

（**【HITL・事前核准】** **【HITL・例外升級】**）。

必要章節：權限等級表（A 自主 / B 需核准 / C 禁止）・ **升級條件**・ 審計要求。

寫法規則 - 用「類別」分級，不逐條列舉。 - 升級條件必須量化，例如「同一任務連續 3 次測試失敗」「預估變更 > 500 行」。 - 預設值：**未列出的動作視為 B（需核准）**。

4.16 Ops Runbook（L9：營運）

目的：部署、監控、告警與事故處置，讓 agent 與人在壓力下也能照做。

必要章節：環境與部署拓模（Mermaid）・ 發布與回滾步驟・ 監控指標與告警門檻（含數字）・ 事故處置步驟・ 資料備份與復原。

寫法規則：每個步驟可複製執行；標明**不可逆**步驟與所需核准（**【HITL・事前核准】**）；所有告警都要有對應處置步驟。agent 對正式環境的操作屬於 C 級（禁止），除非 Autonomy Policy 另有明文。

4.17 Agent Spec 與 Tool Spec（L10：產品內含 agent）

Agent Spec 必要章節：職責與**非職責**・ 輸入輸出（哪些由系統注入、不由模型決定）・ 可用工具・ **護欄**・ 失敗處理・ 評測連結與門檻・ 所用的 agent 模式（見 5.5）。

Tool Spec 必要欄位：`name`・ `description`（寫何時該用、何時不該用）・ `parameters`（用 enum、pattern 收斂）・ `returns`（含錯誤碼）・ `side_effects`・ 是否可逆・ 是否幂等。

寫法規則 - 涉及金額、刪除、對外發送的工具，必須有人工確認關卡。 **【HITL・事前核准】** - 使用者身分等敏感參數由系統覆寫，不信任模型輸出。 - 外部資料（訂單內容、網頁、信件）一律視為**不可信輸入**，其中的指令不得執行。

4.18 Task Spec（執行層：任務單）

目的：規格是長期的，任務單是一次性的工單。好的任務單讓 agent 一次做對。

必要章節：目標（指向規格的 ID）・ **範圍（可修改的路徑）**・ **禁止修改的路徑**・ 參考（規格、範例檔）・ **完成條件（可勾選）**・ 不確定時的處置。

寫法規則 - 範圍用路徑 glob，不用「相關檔案」。 - 完成條件全部可由指令判定。 - 一個任務應能在**單一 PR、≤ 500 行**內完成；否則拆分。 - 任務只能指向 `ready` 的規格。

4.19 Change Request（執行層：變更請求）

目的：當 agent 發現需要改動不屬於自己的資產（他人 repo、共用契約、被依賴的規格）時，不得自行修改，而是提出 CR 交給擁有者。**【HITL・事前核准】**

必要章節：發起者・ 目標資產與擁有者・ 變更內容・ **動機與證據**・ **相容性分析**（是否破壞性）・ 影響範圍（依賴者清單）・ 建議發布順序・ 決議紀錄。

寫法規則：CR 放在**擁有者的 repo**（或契約庫）；決議後轉為擁有者的 Task 或 Feature 變更。範例見 10.5。

4.20 Epic（組織層：跨功能與跨專案計畫）

目的：把一個需要多個功能、多個專案協作的目標，拆成可平行的工作，並規劃依賴與發布順序。

必要章節：目標與成功指標・ **工作分解表**（子項目、擁有者、repo、狀態）・ **依賴圖**・ **介面先行清單**（必須先定案的契約）・ 發布順序與相容性計畫・ 風險與回滾。

寫法規則 - 工作分解表中每個子項目都指向一個 Feature / Task / CR 的 ID。 - 跨專案 Epic 必須置於**共用契約庫**或指定的協調 repo，避免「兩邊都以為對方會寫」。 - Epic 狀態由子項目彙整，**不手動維護進度百分比**。

第 5 章 UML 與 GoF 設計模式的定位

5.1 角色改變：從「描述設計」到「約束設計」

	過去	Agentic 時代
UML	給人看的設計圖，畫完就腐爛	給 agent 與人共用的結構約束；放 repo、可 diff、可被驗證
GoF	工程師的共同詞彙	限制 agent 自行發明架構的詞彙表，同時阻止過度套用

原因：agent 遇到規格空白時最愛填的是過度設計（到處加 Factory、Singleton、抽象層）與結構漂移（同一專案出現三種寫法）。UML 與模式政策正是用來堵這兩個洞。

5.2 保留的 6 種 UML 圖

圖	對應文件	解決的問題	Mermaid 語法
Component / C4	ARCH	模組邊界、依賴方向	flowchart / C4
Class	DOM	實體關係、多重性、聚合	classDiagram
State Machine	SM	合法轉換（最高價值）	stateDiagram-v2
Sequence	FLOW	跨模組呼叫與失敗路徑	sequenceDiagram
ER	C (erd)	資料表與關聯	erDiagram
Deployment	OPS	環境與部署拓模	flowchart

Activity 與 Use Case 圖以 Feature 的 Given / When / Then 取代，不另畫。

圖的三條鐵則（P7） 1. 圖 + 文字約束 + 驗證，三者缺一不可。agent 讀圖不如讀文字可靠（箭頭是呼叫還是依賴？），所以圖旁邊必須附規則與驗證方式。 2. 圖必須與程式碼同 PR 更新（列入 DoD）；能由程式碼產生的（如 import 依賴圖）就用腳本產生。 3. 用測試強制執行，圖才不只是參考。

5.3 架構測試範例（把圖變成可執行的規則）

```
// test/architecture/layer_rule_test.dart — 對應 ARCH-001
import 'dart:io';
import 'package:test/test.dart';

List<String> filesImporting(String dir, List<String> forbidden) {
  return Directory(dir)
    .listSync(recursive: true)
    .whereType<File>()
    .where((f) => f.path.endsWith('.dart'))
    .where((f) {
      final imports = f
        .readAsLinesSync()
        .where((l) => l.trimLeft().startsWith('import '));
      return imports.any((l) => forbidden.any(l.contains));
    })
    .map((f) => f.path)
    .toList();
}

void main() {
  test('ARCH-001 domain 不得依賴 data 或 presentation', () {
    final v = filesImporting('lib/domain', ['/data/', '/presentation/']);
    expect(v, isEmpty, reason: '違反 ARCH-001: $v');
  });

  test('ARCH-002 presentation 不得直接依賴 data', () {
    final v = filesImporting('lib/presentation', ['/data/']);
    expect(v, isEmpty, reason: '違反 ARCH-002: $v');
  });
}
```

也可使用 `import_lint`、`custom_lint` 等套件。重點是每條 ARCH 規則都有對應的失敗測試。

5.4 GoF → Pattern Policy

不要在規格裡教 agent 什麼是 Strategy（它比多數人熟），而是規定在本專案中哪裡用、哪裡禁用。

建議預設政策（Flutter 專案可直接採用，再依專案調整）

情境	模式	備註
資料來源抽象（API / 快取）	Repository	介面放 domain，實作放 data
狀態變化通知 UI	Observer（Riverpod Notifier）	不得自製 Stream 通知機制
多種計價／折扣規則	Strategy	新規則新增類別，不改既有類別
隔離第三方 SDK	Adapter	業務層不得 import SDK
有限狀態與轉換	State（sealed class）	搭配 SM 轉換表
物件建立與依賴	建構子注入	取代 Singleton 與 Service Locator

禁止／限制	規則	理由
Singleton	禁止，改用 provider	難測試、隱藏依賴
Abstract Factory / Builder	需附理由並核准	agent 易過度設計
為「未來」預留的抽象	禁止；至少 2 個具體實作才可抽象	YAGNI
繼承超過 2 層	禁止，改用組合	脆弱基底類別

決策規則：已有範例就沿用；只有一個實作的介面不建立；清單外的模式在 PR 說明「解決什麼具體問題」並等待核准。

Dart 3 讓部分模式簡化，規格應指示 agent 使用現代寫法，把規則交給編譯器：

```
// 取代 State / Visitor:sealed class + 窮盡式 switch
sealed class OrderState {}
class Created extends OrderState {}
class Paid extends OrderState { Paid(this.paidAt); final DateTime paidAt; }
class Shipped extends OrderState { Shipped(this.trackingNo); final String trackingNo; }

String label(OrderState s) => switch (s) {
  Created() => '待付款',
  Paid() => '已付款',
  Shipped() => '已出貨',
  // 少寫任何子類別，編譯器直接報錯（硬度 5）
};
```

Pattern Policy 應補一句：「狀態類型一律用 sealed class，禁止 enum + 多層 if。」

5.5 產品含 agent 時：agent 設計模式

模式	用途	適用
Prompt Chaining	固定步驟串接	流程確定（擷取 → 驗證 → 格式化）
Routing	先分類再分派	客服意圖分流
Orchestrator-Workers	主 agent 拆任務給子 agent	任務無法預先拆解
Evaluator-Optimizer	產出與評審迭代	有明確品質標準
Human-in-the-loop	關鍵動作需人工確認（【HITL・事前核准】）	金流、刪除、對外發送
ReAct	邊推理邊呼叫工具	開放式問題

原則：能用固定流程就不要用自主 agent，前者可預測、可測試。選擇理由寫成 ADR；流程圖上標明「哪一步由模型決定、哪一步由程式決定」。

第 6 章 追溯、交叉參照與變更影響

6.1 追溯鏈



規則 G-6.1：任何一段程式碼都應能回溯到某條需求；任何一條 MUST 需求都應能前進到某個測試。無來源的程式碼視為缺陷（或需補規格）。

6.2 雙向追溯

方向	做法	檢查
需求 → 測試	Feature 的 <code>verify</code> 與場景標示	<code>spec_lint : implemented</code> 的 Feature 其 <code>verify</code> 檔案必須存在
測試 → 需求	測試內 <code>// covers F-012#R1</code>	腳本比對：所有 MUST 需求至少被一個 <code>covers</code> 標記
依賴 → 被依賴	<code>depends_on</code>	<code>spec_lint --impact <ID></code> 列出所有依賴者

6.3 變更影響分析 (Change Impact)

當被依賴的文件（Feature、Contract、SM、INV）要變更時，變更者必須先做影響分析：

- 執行 `dart run tool/spec_lint.dart --impact <ID>`，取得依賴者與參照者清單。
- 判斷變更類型：

類型	定義	處置
相容 (Compatible)	僅新增選用欄位、放寬限制、補充說明	次版號 +1；通知參照者，無需阻擋
破壞 (Breaking)	移除或改名欄位、收緊限制、改變既有行為、改變轉換表	主版號 +1；必須為每個依賴者建立 Task 或 CR；依賴者狀態退回 <code>draft</code> 或標記 <code>needs-review</code>
澄清 (Clarification)	僅修正措辭、不改行為	修訂號 +1；不需通知

- 將結果寫入 PR 描述的「影響範圍」段與 `STATUS.md`。
- 破壞性變更不得與其依賴者的修改混在同一個 PR，除非兩者同屬一個原子任務且已在 Epic 中說明。

6.4 版本與相容性

- Feature、Contract、Pattern Policy 皆使用語意化版本 `MAJOR.MINOR.PATCH`。
- 參照者以 `@MAJOR.MINOR` 表示最低需求版本：`C-CART-API@1.3` 代表「需要 1.3 以上、且主版號仍為 1」。
- 契約的破壞性變更採 `Expand` → `Migrate` → `Contract` 三階段（見 7.2.5），不可一次切換。

變更	版本	例
新增選用欄位、新增錯誤碼（已說明處理方式）	MINOR	409 回應新增 <code>max_available</code>
修正文件措辭、補範例	PATCH	
欄位改名、移除、型別變更、必填化	MAJOR	<code>quantity</code> 由 int 改為物件

6.5 參照衛生規則

規則	說明	檢查
G-6.5a 無懸空參照	<code>depends_on</code> / <code>refs</code> 指向的 ID 必須存在	<code>spec_lint</code>
G-6.5b 無循環依賴	<code>depends_on</code> 形成的圖必須是 DAG	<code>spec_lint</code>
G-6.5c 方向規則	低層不得依賴高層：Feature 可依賴 Contract / SM /	審查

	INV ; Contract 不得依賴 Feature	
G-6.5d 就緒規則	ready 文件不得依賴 draft 文件	spec_lint
G-6.5e 廢止規則	被依賴者 deprecated 時，依賴者須在 14 天內改指向替代文件	審查
G-6.5f 不複製	內文不得複製被參照處的需求文字	審查

遇到循環依賴怎麼辦：兩個功能互相需要時，抽出共同的第三份文件（通常是 Contract 或 SM），兩者都依賴它。

6.6 功能地圖_index.md

docs/04-features/_index.md 是所有功能的目錄與依賴總表，供 agent 快速定位、供人排程。

```
---
id: IDX-features
type: index
status: ready
owner: team-cart
---
| ID | 標題 | 狀態 | 依賴 | 擁有者 | 主要路徑（擁有權） |
| --- | --- | --- | --- | --- | --- |
| F-010 | 加入購物車 | implemented | C-CART-API@1.0 | team-cart | lib/features/cart/add/** |
| F-012 | 修改數量 | implementing | F-010, C-CART-API@1.3, C-STOCK-POLICY@1.0 | team-cart | lib/features/cart/quantity/** |
| F-013 | 庫存檢查 | ready | C-STOCK-POLICY@1.0, C-STOCK-API@1.1 | team-cart | lib/features/cart/stock/** |
| F-020 | 結帳 | ready | F-012@1.2, F-013@1.0, SM-order@1.0 | team-pay | lib/features/checkout/** |
```

規則：「主要路徑」欄即 7.1 的檔案擁有權矩陣；新增功能時必須先在此登記路徑，避免兩個任務改同一批檔案。

第 7 章 協作規範

7.1 同一專案內：多功能、多 agent 並行

7.1.1 前提：先切分，再並行

並行的先決條件是邊界清楚。在發出任務之前，必須確認：

檢查	不通過時
每個任務有互不重疊的可修改路徑	重新切分，或改為序列執行
任務間共用的介面已有契約（介面先行）	先寫契約任務，其餘任務等待
每個任務單獨可測、可合併	拆得更小

7.1.2 介面先行（Interface First）

當功能 A 需要使用功能 B 提供的能力時：

1. 先定義介面（Dart `abstract interface class`、OpenAPI、或 Feature 的「對外介面」段），狀態設為 `ready`。
2. A 依賴介面，不依賴 B 的實作；測試使用 fake / mock。
3. B 負責讓實作滿足介面；兩者各自通過測試後再整合。

7.1.3 檔案擁有權與共用檔案協定

- 每個路徑在同一時間只屬於一個進行中的任務（以 `_index.md` 與任務單的「範圍」為準）。
- 共用檔案（路由表、DI 註冊、l10n、pubspec、barrel file）採以下協定：
 1. 指定一個整合任務專責修改共用檔案；
 2. 其他任務只能在任務描述中提出需要的條目，由整合任務合併；
 3. 或採用自動合併友善的寫法（每功能自己的 provider 檔、`part` 檔、l10n 分檔）降低衝突。
- agent 不得因為「順手」而修改範圍外的檔案；需要時提出 CR 或在 STATUS.md 記錄。

7.1.4 合併順序與衝突處置

情況	規則
有依賴	依 DAG 拓撲順序合併，被依賴者先合
無依賴	先 ready 先合；後合者負責 rebase 並重新跑全部測試
出現衝突	不得用「接受雙方」或刪除對方變更解決；若非機械性衝突，停止並回報。【HITL・例外升級】
測試因他人變更而失敗	先確認是自己的問題還是上游問題；上游問題→在 STATUS.md 記錄並通知擁有者

7.1.5 agent 之間如何溝通

Agent 之間不直接對話，所有協調經由可持久化、可審計的文件：

需要傳達	管道
進度、阻擋	STATUS.md
要求他人變更	CR（同 repo 可用任務單的 requests 欄）
規格缺漏	STATUS.md 的「待釐清」，由人工回答後寫回規格。【HITL・事中澄清】
介面變更	修改契約 + 版本遞增 + 影響分析

7.2 跨專案協作

7.2.1 角色與資產擁有權

角色	定義
Provider（提供者）	擁有並維護某份契約的專案（如 <code>shop-api</code> 擁有購物車 API 的實作）

Consumer (消費者)	依賴該契約的專案 (如 <code>shop-app</code>)
Contract Owner	共用契約庫 (<code>platform-contracts</code>) 中該契約的負責團隊

規則 G-7.2a：每個資產有且僅有一個擁有者。非擁有者不得直接修改，只能提出 CR。 **【HITL・事前核准】**

7.2.2 共用契約庫 (單一事實來源)

```
platform-contracts/      # 獨立 repo
├── AGENTS.md
├── contracts/
│   ├── cart/openapi.yaml # C-CART-API
│   ├── stock/openapi.yaml # C-STOCK-API
│   └── events/*.json
├── epics/
│   └── E-2026-014-stock-aware-cart.md
├── change-requests/
│   └── CR-0007.md
├── compat-matrix.md      # 各專案支援的契約版本
└── CHANGELOG.md
```

- 契約以 `git tag` 發布 (如 `cart-api/v1.3.0`)。各專案以版本依賴，不得依賴 `main`。
- `compat-matrix.md` 記錄「哪個專案的哪個版本支援哪些契約版本」。

7.2.3 跨專案的參照與依賴

- 使用 `project:ID` 語法：`[[platform-contracts:C-CART-API@1.3]]`。
- 各專案 `docs/05-contracts/` 不複製契約本文，只放鎖定版本的引用 (`contracts.lock`) 與生成碼：

```
# contracts.lock (由工具更新，不手改)
platform-contracts:
  cart-api: 1.3.0
  stock-api: 1.1.2
```

7.2.4 變更請求 (CR) 流程

```
1 Consumer agent 發現需要契約變更
  | (不得自行修改)
  ▼
2 在 platform-contracts 建立 CR (動機、證據、相容性分析、建議方案)
  ▼
3 Contract Owner (人) 審查 —否決→ 回覆原因，結束
  |核准
  ▼
4 Owner 或其 agent 修改契約、遞增版本、發布 tag
  ▼
5 Provider 與 Consumer 各自建立 Task，針對**新版契約**實作與測試
  ▼
6 契約測試通過 → 依發布順序 (7.2.5) 上線
```

【HITL・事前核准】 步驟 3 由 Contract Owner (人) 審查與決議；agent 不得代為核准自己提出的 CR。

7.2.5 發布順序與相容性 (Expand → Migrate → Contract)

跨專案變更不得要求「兩邊同時上線」。採三階段：

階段	動作	相容性要求
Expand 擴充	Provider 先上線同時支援新舊的版本	舊 Consumer 不受影響
Migrate 遷移	Consumer 逐一升級到新契約	新舊並存
Contract 收斂	所有 Consumer 升級後，Provider 移除舊行為	需確認 <code>compat-matrix.md</code> 已無舊版 Consumer

行動 App 特別注意：App 版本無法強制更新，舊版會長期存在。收斂階段必須以實際版本分佈數據為依據，不得憑感覺。

7.2.6 消費者驅動契約測試

- Consumer 把「我依賴哪些欄位與行為」寫成 **consumer contract** (如 `Pact`)，放進契約庫。
- Provider 的 CI 必須驗證所有 consumer contract；任何一個失敗即不得發布。

- 這使 Provider 在修改前就知道誰會被影響，而不是上線後才發現。

7.2.7 每個 repo 的 AGENTS.md 必須有的跨專案段落

跨專案規則

- 本專案擁有：`Lib/\*\*` (App)。契約由 platform-contracts 擁有。
- 不得修改其他 repo 或 `contracts.lock` 以外的契約引用。
- 需要契約變更 → 在 platform-contracts 建立 CR (模板：change-requests/_template.md)，然後在 STATUS.md 的「阻擋」登記 CR 編號並停止相關工作。
- 契約版本以 contracts.lock 為準；升級版本須由任務單授權。

7.2.8 跨專案進度與交接

- Epic 是跨專案的唯一進度來源；各 repo 的 STATUS.md 只記錄本 repo 的事。
 - 每個子項目在 Epic 的工作分解表中對應一個「project:ID」，由各 repo 的任務完成後回寫狀態（或由腳本彙整）。
-

第 8 章 Agent 執行協議

本章直接寫給 agent。agent 在每個 session 開始時必須遵守。

8.1 載入順序 (Context Loading)

不要一次讀完所有文件。依下列順序**按需載入**：

順序	載入	何時
1	AGENTS.md	永遠
2	STATUS.md	永遠
3	被指派的 tasks/T-xxxx.md	永遠
4	任務指向的 Feature（及其 depends_on、refs 一層）	開工前
5	該 Feature 參照的 SM、Contract、UX、INV	需要時
6	pattern-policy.md、architecture.md	新增類別、模組、依賴時
7	autonomy-policy.md	遇到 B / C 級動作或需升級時
8	相關 ADR	想要質疑或更動既有結構時

規則：只載入一層依賴。需要更深一層時，先確認它確實影響本任務。

8.2 開工前檢查 (Preflight)

在寫任何程式碼前，逐項確認；**任何一項不通過就停止並回報**（【HITL・例外升級】）：

- ☐ 任務指向的規格狀態為 ready 或 implementing
- ☐ 規格的 depends_on 全部已 implemented，或其介面已為 ready（可用 fake 開發）
- ☐ 任務的「範圍」與其他進行中的任務**無重疊**（對照 index.md）
- ☐ 執行 AGENTS.md 的建置與測試指令，**基線是綠的**（否則先回報，不要在紅燈上疊加）
- ☐ 所有「完成條件」都能以指令判定
- ☐ 規格中沒有阻擋性的開放問題

8.3 執行中規則

- 先寫測試再實作**（或至少同時）：先將場景轉成失敗的測試，確認它因正確的原因失敗。
- 小步提交**：每個場景通過就提交一次，訊息含 ID（F-012: S1 passing）。
- 只改範圍內的檔案**。需要範圍外的修改 → 停止，登記於 STATUS.md 或提 CR。
- 沿用既有寫法**：新增檔案前，先找 Pattern Policy 指定的範例檔，模仿其結構與命名。
- 不新增依賴套件**（B 級），除非任務明確授權。（【HITL・事前核准】）
- 不得為了讓測試通過而**：刪除測試、放寬斷言、加 skip、修改規格。若認為規格有誤，走 8.4（【HITL・事中澄清】）。
- 同步更新**：類別圖、轉換表、Golden 圖、verify 路徑。

8.4 不確定時的決策樹



原則：「**停止並回報**」永遠是合法且被鼓勵的選項。（【HITL・事中澄清】／【HITL・例外升級】）猜錯並讓錯誤進入主幹的成本，遠高於暫停一

次。

8.5 升級條件（需停止並回報人工）

本節整節屬 **【HITL・例外升級】**。

條件	說明
同任務連續 3 次測試失敗且原因不明	避免無限嘗試
規格與既有程式碼衝突	不得自行裁決誰對
預估變更 > 500 行	先提交拆分計畫
涉及金流、權限、個資、資料遷移、不可逆操作的規格有任何疑義	風險類別一律升級
發現安全問題（金鑰洩洩、注入、越權）	立即回報，不得公開細節於一般 PR 描述
需要存取正式環境	C 級，禁止

8.6 完工與交接

完成的定義（DoD）——全部滿足才可標記完成：

1. 任務單所有完成條件已勾選且可重現。
2. `dart analyze --fatal-infos` 零輸出；`dart format` 無差異。
3. 相關測試全過；新增行為皆有測試，並標 `covers`。
4. 架構測試（`test/architecture`）通過。
5. 規格同步：Feature 狀態、`verify` 路徑、圖與轉換表已更新。
6. `STATUS.md` 已更新（進行中／完成／待釐清）。
7. PR 描述完整（見 8.7）。

8.7 PR 描述模板

【HITL・事後審查】 PR 由人審查。以下模板的目的，是讓審查者能在最短時間內判斷「需求是否被滿足」與「哪些是 agent 自行決定的」。

```
## 任務
T-0231 · 實作 F-012 R1-R3 (S1-S3)

## 變更摘要
- 新增 QuantityStepper 元件與 CartQuantityNotifier
- 新增 3 個 widget test、1 個整合測試

## 規格對應
| 需求 | 場景 | 測試 |
| --- | --- | --- |
| F-012#R1 | S1, S2 | quantity_test.dart: 'S1', 'S2' |
| F-012#R3 | S3 | quantity_test.dart: 'S3' |

## 自主決定（規格未寫、可逆的小決定）
- debounce 計時器放在 Notifier 內而非 Widget（利於測試）

## 待釐清（已寫入 STATUS.md）
- Q-04：R4 數量降至 0 的詢問文案

## 影響範圍
- `spec_lint --impact F-012`：F-014 (refs)、F-020 (depends_on) – 皆為相容變更，無需動作

## 檢查
- [x] analyze 零警告 - [x] 測試全過 - [x] 架構測試 - [x] STATUS 已更新
```

第 9 章 品質閘門與自動化

9.1 CI 閘門

閘門	內容	失敗即阻擋合併
G1 靜態分析	<code>dart analyze --fatal-infos</code> 、 <code>dart format</code> <code>--set-exit-if-changed</code>	✓
G2 單元／Widget 測試	<code>flutter test --coverage</code>	✓
G3 架構測試	<code>flutter test test/architecture</code>	✓
G4 規格檢查	<code>dart run tool/spec_lint.dart</code>	✓
G5 追溯檢查	所有 <code>ready</code> 以上 Feature 的 MUST 需求被 <code>covers</code> 標記	✓
G6 契約檢查	schema 驗證 + 契約測試 (Provider 端含所有 consumer contract)	✓
G7 Golden 測試	UI 狀態矩陣的截圖比對	✓
G8 Eval	含 LLM／agent 時，eval 通過率 ≥ 門檻	✓
G9 依賴與授權	新增套件需在 PR 標記核准 (【HITL・事前核准】)	✓
G10 變更範圍	變更檔案是否在任務單「範圍」內、有無觸碰禁區	✓

9.2 spec_lint：規格檢查工具（參考實作）

功能：檢查 front-matter 必填欄位、`status` 合法性、`id` 唯一、`depends_on` / `refs` 指向存在、循環依賴、`ready` 不依賴 `draft`、`implemented` 的 `verify` 檔案存在；並支援 `--impact <ID>` 列出依賴者。

```
// tool/spec_lint.dart
// 用法: dart run tool/spec_lint.dart          檢查全部
//       dart run tool/spec_lint.dart --impact F-012 影響分析
import 'dart:io';

const requiredKeys = ['id', 'type', 'status', 'owner'];
const validStatus = {'draft', 'ready', 'implementing', 'implemented', 'deprecated'};
const activeStatus = {'ready', 'implementing', 'implemented'};

typedef FrontMatter = Map<String, List<String>>;

FrontMatter parseFrontMatter(String src) {
  final m = RegExp(r'^---\r?\n([\s\S]*?)\r?\n---').firstMatch(src);
  final result = <String, List<String>>{};
  if (m == null) return result;
  for (final line in m.group(1)!.split('\n')) {
    final i = line.indexOf(':');
    if (i <= 0 || line.startsWith(' ') || line.startsWith('#')) continue;
    final key = line.substring(0, i).trim();
    var value = line.substring(i + 1).split(' #').first.trim();
    if (value.startsWith '[' && value.endsWith ']') {
      value = value.substring(1, value.length - 1);
      result[key] = value
        .split(',')
        .map((e) => e.trim())
        .where((e) => e.isNotEmpty)
        .toList();
    } else {
      result[key] = [value];
    }
  }
  return result;
}

String baseId(String ref) => ref.split('@').first.split('#').first;

void main(List<String> args) {
  final meta = <String, FrontMatter>{};
  final paths = <String, String>{};
  final errors = <String>[];

  final files = Directory('docs')
    .listSync(recursive: true)
    .whereType<File>()
```

```

        .where((f) => f.path.endsWith('.md'));

for (final f in files) {
    final fm = parseFrontMatter(f.readAsStringSync());
    if (fm.isEmpty) continue; // 無 front-matter 者略過 (如 README)
    for (final k in requiredKeys) {
        if ((fm[k] ?? const []).isEmpty || fm[k]!.first.isEmpty) {
            errors.add('${f.path}: 缺少必填欄位 "$k"');
        }
    }
    final id = fm['id']?.firstOrNull;
    if (id == null || id.isEmpty) continue;
    if (meta.containsKey(id)) {
        errors.add('${f.path}: id 重複 "$id" (另見 ${paths[id]}) ');
    }
    meta[id] = fm;
    paths[id] = f.path;
    final st = fm['status']?.firstOrNull;
    if (st != null && !validStatus.contains(st)) {
        errors.add('${f.path}: status "$st" 不合法');
    }
}

// --impact : 列出依賴者與參照者後結束
final impactIdx = args.indexOf('--impact');
if (impactIdx >= 0 && impactIdx + 1 < args.length) {
    final target = args[impactIdx + 1];
    print('== 影響分析: $target ==');
    for (final e in meta.entries) {
        final dep = (e.value['depends_on'] ?? const []).map(baseId).contains(target);
        final ref = (e.value['refs'] ?? const []).map(baseId).contains(target);
        if (dep) print(' [depends_on] ${e.key} (${paths[e.key]})');
        if (ref) print(' [refs]      ${e.key} (${paths[e.key]})');
    }
    return;
}

for (final e in meta.entries) {
    final id = e.key, fm = e.value, path = paths[id]!;
    final status = fm['status']?.firstOrNull;

    // 參照存在性 (跨專案參照含 ":" 者交由跨 repo 檢查)
    for (final key in ['depends_on', 'refs']) {
        for (final raw in fm[key] ?? const <String>[]) {
            final ref = baseId(raw);
            if (ref.contains(':')) continue;
            if (!meta.containsKey(ref)) errors.add('$path: $key 指向不存在的 "$raw"');
        }
    }

    // ready 不得依賴 draft
    if (status != null && activeStatus.contains(status)) {
        for (final raw in fm['depends_on'] ?? const <String>[]) {
            final dep = baseId(raw);
            if (dep.contains(':')) continue;
            if (meta[dep]?['status']?.firstOrNull == 'draft') {
                errors.add('$path: $status 的文件不得依賴 draft 的 "$dep"');
            }
        }
    }

    // Feature 驗證規則
    if (fm['type']?.firstOrNull == 'feature' &&
        status != null &&
        activeStatus.contains(status)) {
        final verify = fm['verify'] ?? const <String>[];
        if (verify.isEmpty) errors.add('$path: $status 的 feature 必須宣告 verify');
        if (status == 'implemented') {
            for (final p in verify) {
                if (!File(p).existsSync() && !Directory(p).existsSync()) {
                    errors.add('$path: verify 路徑不存在 "$p"');
                }
            }
        }
    }
}

// 循環依賴 (DFS)
final visiting = <String>{}, done = <String>{};
void visit(String id, List<String> stack) {
    if (done.contains(id)) return;
    if (!visiting.add(id)) {
        errors.add('循環依賴: ${[...stack, id].join(' -> ')}');
        return;
    }
}

```

```

    }
    for (final raw in meta[id]?['depends_on'] ?? const <String>[]) {
        final dep = baseId(raw);
        if (meta.containsKey(dep)) visit(dep, [...stack, id]);
    }
    visiting.remove(id);
    done.add(id);
}

for (final id in meta.keys) {
    visit(id, []);
}

if (errors.isEmpty) {
    print('spec_lint: OK (${meta.length} 份文件) ');
} else {
    errors.forEach(stderr.writeln);
    stderr.writeln('spec_lint: ${errors.length} 個問題');
    exit(1);
}
}

```

這是參考實作，使用前請依團隊的目錄與欄位調整，並以故意製造的錯誤（缺欄位、循環依賴、懸空參照）驗證它真的會失敗。一個從未失敗過的檢查，不能算被驗證過。

9.3 追溯檢查（`covers` 標記）

最簡做法：在 CI 以腳本比對 Feature 的 `R<n>` 與測試中的 `covers F-xxx#R<n>`。

```

# 列出 F-012 中尚未被任何測試 covers 的 MUST 需求（概念示範）
for r in $(grep -oE '^ R[0-9]+ \ (MUST\)' docs/04-features/F-012-*.md | grep -oE 'R[0-9]+'); do
    grep -rq "covers.*F-012#$r" test/ integration_test/ || echo "未覆蓋：F-012#$r"
done

```

9.4 Eval 閘門（含 LLM/agent 的產品）

- Eval 集納入版本控管，與 prompt、工具定義同 PR 變更。
- 任何以下變更皆須重跑 eval：模型版本、系統提示、工具描述或參數、檢索設定。
- 通過率低於門檻 → 阻擋合併；不得以降低門檻或刪除案例作為通過手段（需經人工核准並記錄原因）。【HITL・事前核准】

第 10 章 實戰範例（讀者是 Agent）

本章五個範例共用同一個虛構產品 **ShopLite**（Flutter 行動 App，含購物車、結帳），由淺入深：

範例	範圍	展示重點
10.1	單一專案	從零建立 L0-L3，agent 第一個 session 怎麼讀、怎麼做
10.2	單一功能	一份完整 Feature Spec → Task → agent 實作過程
10.3	多功能並行	三個 agent 同時開工：切分、介面先行、檔案擁有權
10.4	多功能互相參照	引用、版本鎖定、變更影響分析
10.5	多專案協作	App + API + 共用契約庫：CR、Epic、發布順序

每個範例都以「Agent 讀到什麼 → 判斷與行動 → 產出」呈現，因為這才是規格書真正被使用的方式。

10.1 單一專案：建立 ShopLite 的規格基礎

情境

新專案 **shoplite**（Flutter 3.x / Dart 3）。人類 PM 與技術主管在第一週建立 L0-L3 的最小集合，之後所有 agent 皆依此工作。

檔案一：AGENTS.md（≤ 100 行）

```
# AGENTS.md

## 專案一句話
ShopLite：跨平台電商 App（Flutter 3.x / Dart 3）。後端 REST API 由 shop-api 專案提供。

## 指令（可直接複製）
- 安裝：`flutter pub get`
- 分析：`dart analyze --fatal-infos`
- 格式：`dart format --set-exit-if-changed .`
- 測試：`flutter test --coverage`
- 架構測試：`flutter test test/architecture`
- 規格檢查：`dart run tool/spec_lint.dart`
- 產生程式碼：`dart run build_runner build -d`

## 完成的定義（DoD）
1. analyze 零輸出、format 無差異
2. 相關測試與架構測試全過；新增行為標 `covers <ID>`
3. 規格同步（狀態、verify、圖、轉換表）
4. STATUS.md 已更新
5. PR 依 `.github/pull_request_template.md` 填寫

## 架構規則（MUST）
- 分層 presentation → domain ← data（ARCH-001/002）；domain 不得 import Flutter
- 狀態管理只用 Riverpod；金額只用 `Money`（INV-003）
- UI 字串一律走 l10n；顏色與間距一律用 token，不寫死
- 新模式、新套件、新模組先讀 docs/03-architecture/pattern-policy.md，B 級需核准

## 禁區（NEVER）
- 不得修改：`lib/generated/**`、`.github/workflows/**`、`docs/05-contracts/**`
- 不得提交金鑰；不得刪除或跳過既有測試來讓測試通過
- 不得存取正式環境

## 不確定時
- 規格矛盾或缺漏且影響行為／金額／權限／資料 → 停止；寫入 STATUS.md「待釐清」；回報
- 屬可逆的小細節 → 自行決定，記錄於 PR「自主決定」段
- 需改範圍外檔案或他人資產 → 停止；提 CR 或登記 STATUS.md

## 文件導覽
意圖 docs/01-intent · 術語與不變量 docs/02-domain · 架構與模式 docs/03-architecture
功能 docs/04-features/_index.md · 契約 docs/05-contracts · UX docs/06-ux
權限 docs/08-policy/autonomy-policy.md · 進度 STATUS.md · 準則 docs/00-guideline
```

檔案二：docs/01-intent/PI-001-shopping-cart.md

```
---
```

```
id: PI-001
type: intent
title: 購物車
status: ready
owner: pm-lin
version: 1.0.0
---
```

目標使用者

手機優先的一般消費者；單手操作；網路時好時壞（行動網路、電梯、捷運）。

要解決的問題

使用者加入商品後離開 App 再回來，購物車內容不應消失，且數量調整要即時有回饋。

成功指標

指標	目標	量測方式
加入購物車 → 抵達結帳頁完成率	≥ 65%	事件 `cart_to_checkout`
購物車頁首屏載入 P95	≤ 800 ms	APM, Pixel 6 / iPhone 12
數量調整失敗後使用者可復原率	≥ 95%	事件 `qty_error_recovered`

非目標 (Non-goals)

- 不做跨裝置即時同步（第二階段）
- 不做優惠券疊加計算
- 不改動結帳流程（另屬 PI-002）

已決定事項（不得重新討論）

- 購物車以**後端為準**，本地僅作快取（見 ADR-0004）
- 每品項數量上限 99（INV-001）

開放問題

#	問題	負責人	期限
Q-1	缺貨商品是否自動移除？	pm-lin	2026-10-20

檔案三：初始 `STATUS.md`

```
# STATUS (最後更新: 2026-10-07 by human tech-lead)
```

進行中

(無)

已完成

- 專案骨架、CI (G1-G4)、AGENTS.md、PI-001、INV-001..003、ARCH-001..002、ADR-0004

待釐清 (需人工回答)

- Q-1 (PI-001)：缺貨商品是否自動移除？負責人 pm-lin，期限 2026-10-20。影響 F-013。

已知問題 / 技術債

(無)

下一步建議

- 撰寫 F-010 加入購物車 (M 級)

Agent 視角：第一個 session

步驟	Agent 讀到	判斷與行動
1	AGENTS.md	記下指令與禁區；發現 docs/05-contracts/** 為禁區 → 日後契約問題不自行改
2	STATUS.md	無進行中任務；Q-1 尚未回答 → 與缺貨相關的行為不得自行決定。【HITL・事中澄清】
3	被指派「撰寫 F-010 草稿」	規格尚不存在 → 狀態只能是 draft；agent 的產出是草稿與提問，不是程式碼
4	執行 flutter test、dart analyze	基線為綠 → 可開始（Preflight 通過）
5	產出 F-010 草稿，在「開放問題」寫入 Q-1 的引用	狀態保持 draft，PR 標示「待 pm-lin 審核」。【HITL・事後審查】

這個範例要傳達的：新專案的 agent 不是一上來就寫程式，而是先確認哪些規格已 `ready`。沒有 `ready` 的規格，就沒有實作任務。

10.2 單一功能：F-012「購物車修改數量」

情境

PI-001 已 **ready**，F-010 已 **implemented**。現在要實作「修改數量」。這是 M 級：需要 Feature Spec + Task；有畫面 → 需要 UX 元件規格；呼叫 API → 引用 Contract；注入庫存政策 → 引用內部介面契約。

文件一：**docs/04-features/F-012-cart-quantity.md**

```
---
id: F-012
type: feature
title: 購物車：修改數量
status: ready
owner: team-cart
version: 1.2.0
depends_on: [F-010, C-CART-API@1.3, C-STOCK-POLICY@1.0]
refs: [PI-001, INV-001, INV-002, ADR-0004, UX-quantity-stepper]
verify: [test/features/cart/quantity/quantity_test.dart, integration_test/cart_flow_test.dart]
updated: 2026-10-07
---

# F-012 購物車：修改數量

## 目的與範圍
讓使用者在購物車頁以 +/- 調整單一 SKU 的數量。對應 [PI-001]。

## 需求
- R1 (MUST)：數量範圍為 1..99 的整數 ([INV-001])。
- R2 (MUST)：數量變更後 300 ms 內送出請求 (debounce)；請求期間 +/- 必須 disabled。
- R3 (MUST)：請求失敗時，數量必須回復為變更前的值，並顯示 Snackbar「更新失敗，請重試」。
- R4 (SHOULD)：數量降至 0 時不直接刪除，改為詢問是否移除。
- R5 (MUST)：可用上限 = min(99, `StockPolicy.maxQuantityFor(info)`)；達上限時「+」必須 disabled。

## 場景
### S1：正常增加 (驗證 R1、R2)
Given 購物車有 SKU-A 數量 2，庫存 50
When 使用者點擊「+」
Then 畫面顯示 3；300 ms 後呼叫 updateCartItemQuantity(sku=SKU-A, quantity=3)

### S2：超過 99 上限 (驗證 R1)
Given SKU-A 數量為 99，庫存 500
When 使用者點擊「+」
Then 數量維持 99；「+」為 disabled；不發送請求

### S3：網路失敗 (驗證 R3)
Given SKU-A 數量為 2；API 回傳 500
When 使用者點擊「+」
Then 畫面短暫顯示 3；失敗後回復為 2；顯示 Snackbar「更新失敗，請重試」

### S4：庫存低於 99 (驗證 R5)
Given SKU-B 數量為 4，庫存 5 (`maxQuantityFor` 回傳 5)
When 使用者點擊「+」兩次 (間隔 > 300 ms)
Then 第一次成功為 5；第二次「+」為 disabled；僅送出 1 次請求

## 邊界與錯誤
| 情況 | 預期行為 |
| --- | --- |
| 快速連點 5 次 (< 300 ms 內) | 只送出最後一次的值 |
| 離線 | +/- 為 disabled；顯示離線提示 |
| API 回 409 (庫存不足) | 數量調整為回應的 `max_available`；顯示提示 |
| API 回 401 | 走全域登入流程 (不在本功能範圍) |
| 請求逾時 (> 10 s) | 視同失敗，依 R3 處理 |

## 非功能
- 按下到畫面變化 ≤ 100 ms
- 觸控目標 ≥ 48×48 dp；TalkBack / VoiceOver 可朗讀「增加數量」「減少數量」

## 介面與資料 (僅參照)
- API：updateCartItemQuantity` ([C-CART-API@1.3])
- 庫存政策：StockPolicy` ([C-STOCK-POLICY@1.0])，由 DI 注入
- UI 元件：[UX-quantity-stepper]

## 超出範圍
- 不處理優惠券重新計算 (見 PI-001 非目標)
- 不處理跨裝置同步

## 驗證
- Widget test：test/features/cart/quantity/quantity_test.dart` (S1–S4)
- 整合：integration_test/cart_flow_test.dart`
- Golden：見 [UX-quantity-stepper]

## 開放問題
- R4 的詢問文案待 pm-lin 確認 (不阻擋 R1–R3、R5；R4 為 SHOULD)
```

文件二：**tasks/T-0231.md**

```

---
id: T-0231
type: task
title: 實作 F-012 R1-R3、R5 (S1-S4)
status: ready
owner: tech-lead
depends_on: [F-012]
---

## 目標
實作 F-012 的 R1、R2、R3、R5，通過 S1-S4。*R4 不在本任務。*

## 可修改範圍
- lib/features/cart/quantity/**
- test/features/cart/quantity/**
- test/goldens/quantity_stepper/**

## 禁止修改
- lib/core/network/** (共用層)
- docs/05-contracts/** (契約變更需 CR)
- lib/app/router.dart、lib/app/providers.dart (共用檔案，見 T-0234)

## 參考
- 規格：F-012；元件：UX-quantity-stepper；INV-001
- 模仿範例：lib/features/cart/add/add_to_cart_notifier.dart (Notifier 寫法)
- 介面：lib/domain/stock/stock_policy.dart (已存在，測試使用 FakeStockPolicy)

## 完成條件
- [ ] `flutter test test/features/cart/quantity` 全過
- [ ] `dart analyze --fatal-infos` 無輸出
- [ ] Golden 四張 (default / at-max / loading / error) 已產生並提交
- [ ] 每個測試標 `covers F-012#Rn`
- [ ] STATUS.md 已更新

## 不確定時
R4 若有疑義，不要實作，寫入 STATUS.md「待釐清」。

```

Agent 視角：實作過程

階段	Agent 讀到	判斷與行動	產出
載入	AGENTS.md → STATUS.md → T-0231 → F-012 (一層依賴：C-CART-API、C-STOCK-POLICY、UX 元件)	不載入 F-010 的全文，只確認其為 <code>implemented</code>	—
Preflight	F-012 <code>ready</code> ；C-STOCK-POLICY <code>ready</code> ；範圍與進行中任務無重疊；基線測試為綠	通過	—
轉測試	S1-S4	先寫 4 個失敗的測試，各標 <code>covers</code> ；確認失敗原因是「元件不存在」而非語法錯誤	<code>quantity_test.dart</code> (紅)
實作 S1	R1、R2	依範例檔建立 <code>CartQuantityNotifier</code> ； <code>debounce 300 ms</code>	S1 綠，提交 F-012: S1 passing
實作 S2、S4	R1、R5	<code>max = min(99, policy.maxQuantityFor(info))</code> ；達上限 + <code>disabled</code>	S2、S4 綠
實作 S3	R3	以樂觀更新+失敗回復；SnackBar 文案走 l10n	S3 綠
遇到疑義	R4 文案未定	這是 SHOULD 且不在本任務 → 不實作；STATUS.md「待釐清」新增 Q-04。【HITL・事中澄清】	STATUS.md 更新
範圍外需求	發現路由需新增購物車頁入口	屬共用檔案、禁止修改 → 在 STATUS.md 記錄「需 T-0234 加入路由」，不動手	—
完工	DoD 7 項	全過； <code>spec_lint --impact F-012</code> 列出 F-014 (refs)、F-020 (depends_on)，本次為相容變更	PR 依 8.7 模板

文件三（節錄）：`docs/06-ux/components/UX-quantity-stepper.md` 狀態矩陣

```
## 狀態矩陣
| 狀態 | 外觀 | 行為 |
| ---|---|---|
| default | 按鈕使用 `color.primary` | 可點擊 |
| at-max | 「+」disabled，透明度 38% | 不觸發 onChanged |
| at-min (R4 SHOULD 實作前) | 「-」disabled | 不觸發 onChanged |
| loading | 中央數字換為 16 px 轉圈 | 兩側按鈕 disabled |
| error | 數字下方 `color.error` 提示文字 | 3 秒後自動消失 |
| offline | 兩側按鈕 disabled | 顯示離線圖示 |

## 規格
- 觸控目標 ≥ 48×48 dp；對比 ≥ 4.5:1
- Semantics：「增加數量」「減少數量」；數字讀出「目前數量 N」
## 驗證
- Golden：default / at-max / loading / error / offline 五張
```

這個範例要傳達的：一份 **ready** 的 Feature + 範圍明確的 Task，使 agent 不需要猜；它**主動不做** R4 與路由修改，並把問題寫進 STATUS.md。

10.3 多功能並行：三個 agent 同時開工

情境

本週要同時完成 F-012（修改數量）、F-013（庫存檢查）、F-014（購物車角標）。每個功能分配給不同 agent。人類主管用本章的規則先切分。

步驟 1：介面先行——先定案內部契約 C-STOCK-POLICY

F-012 需要「某 SKU 最多可買幾個」，F-013 負責提供。若兩者同時各自發明，必然衝突，因此**先寫契約並設為 ready**：

```
---
id: C-STOCK-POLICY
type: contract
title: 庫存政策介面 (內部)
status: ready
owner: team-cart
version: 1.0.0
---

# C-STOCK-POLICY 1.0

```dart
// lib/domain/stock/stock_policy.dart
abstract interface class StockPolicy {
 /// 取得 SKU 的庫存資訊；離線時回傳快取，若無快取回傳 StockInfo.unknown。
 Future<StockInfo> check(Sku sku);

 /// 依庫存資訊回傳可購買的最大數量 (>= 0)。unknown 時回傳 99。
 int maxQuantityFor(StockInfo info);
}

class StockInfo {
 const StockInfo({required this.available, this.lowStock = false});
 const StockInfo.unknown() : available = null, lowStock = false;
 final int? available;
 final bool lowStock;
}
```

## 規則
- `maxQuantityFor` 不得回傳負值，也不得超過 `available`。
- 實作不得拋出例外；失敗以 `StockInfo.unknown()` 表示。
## 驗證
- 契約測試：`test/domain/stock/stock_policy_contract_test.dart`（任何實作皆須通過）
```

步驟 2：功能地圖與檔案擁有權（_index.md 節錄）

| ID | 標題 | 狀態 | 依賴 | 擁有者（任務） | 可修改路徑 |
|-------|------|-------|------------------------------------|---------|--|
| F-012 | 修改數量 | ready | C-STOCK-POLICY@1.0, C-CART-API@1.3 | T-0231 | <code>lib/features/cart/quantity/**</code> |
| F-013 | 庫存檢查 | ready | C-STOCK-POLICY@1.0, C- | T-0232 | <code>lib/features/cart/stock/**</code> 、 <code>lib/data/stock/**</code> |

| | | | | | |
|-------|---------------|-------|---------------|---------------|---|
| | | | STOCK-API@1.1 | | |
| F-014 | 購物車角標 | ready | F-010 | T-0233 | lib/features/cart/badge/** |
| — | 整合：路由、DI、l10n | ready | T-0231..0233 | T-0234 | lib/app/router.dart、lib/app/providers.dart、lib/l10n/*.arb |

檢查：四個任務的路徑互不重疊 ✓ 介面已 ready ✓ DAG：T-0231、T-0232 依賴契約而非彼此 ✓

步驟 3：各 agent 的視角

| Agent | 任務 | 怎麼用規格 | 與他人的關係 |
|---------|----------------|---|-------------------------|
| Agent-A | T-0231 (F-012) | 以 FakeStockPolicy 開發 S4；不 import lib/features/cart/stock/** | 依賴契約，不依賴 Agent-B 的實作 |
| Agent-B | T-0232 (F-013) | 實作 StockPolicy，通過契約測試；以 mock 的 C-STOCK-API 回應開發 | 不知道 Agent-A 何時完成，也不需要知道 |
| Agent-C | T-0233 (F-014) | 實作角標；需要總數量時只讀既有的 cartProvider | 不涉及庫存 |
| Agent-D | T-0234 (整合) | 等 A、B、C 的 PR 合併後統一註冊路由、provider、l10n | 唯一能改共用檔案者 |

步驟 4：衝突情境與處置

情境：Agent-A 在實作時發現需要新增一個 l10n 字串 (cart_update_failed)。

| 錯誤做法 | 正確做法 (依 7.1.3) |
|---------------------------------|--|
| 直接修改 lib/l10n/app_zh.arb (共用檔案) | 在 PR 描述與 STATUS.md 登記「需新增 cart_update_failed：『更新失敗，請重試』」，由 T-0234 統一加入；本功能暫以測試用的 fake 文案通過 |
| 為通過測試而硬編碼字串 | 同上；AGENTS.md 禁止寫死 UI 字串 |

情境：Agent-A 的 PR 先合併，Agent-B 的 PR 隨後出現 rebase 衝突 (兩邊都動了 pubspec.yaml)。

| 規則 (7.1.4) | 動作 |
|--------------|---|
| 後合者負責 rebase | Agent-B rebase；若衝突非機械性 (兩邊新增了不同套件) → 套件屬 B 級，須回報人工，不得自行「兩邊都保留」後送出。【HITL・事前核准】 |

步驟 5：合併順序

C-STOCK-POLICY (ready) 

先 ready 先合；T-0234 最後。

這個範例要傳達的：並行的關鍵不在 agent 多聰明，而在切分前就把介面與檔案範圍定好。三個 agent 彼此不對話，只透過契約、擁有權矩陣與 STATUS.md 協調。

10.4 多功能互相參照：F-020「結帳」與變更影響

情境

F-020 結帳需要重用數量調整 (F-012)、庫存有效性 (F-013)、訂單狀態機 (SM-order)。接著 F-012 的 R3 發生破壞性變更，看規則如何保護 agent 不踩雷。

文件：F-020-checkout.md (節錄)

```
---
id: F-020
type: feature
title: 結帳
status: ready
owner: team-pay
version: 1.0.0
```

```

depends_on: [F-012@1.2, F-013@1.0, SM-order@1.0, C-ORDER-API@1.0]
refs: [INV-002, INV-003, PI-002]
verify: [test/features/checkout/checkout_test.dart]
---
# F-020 結帳

## 需求
- R1 (MUST): 進入結帳前，購物車內每個 CartItem 必須通過 [[F-013#R2]] 的「庫存有效」判定；
  任一不通過時，不得進入結帳，並列出不通過的 SKU。
- R2 (MUST): 建立訂單後，訂單狀態必須為 `Created`，且之後的轉換**完全依照** [[SM-order]]。
- R3 (MUST): 訂單金額以 `Money` 計算 ([[INV-003]])，不得使用浮點數。
- R4 (MUST): 結帳頁修改數量的行為**必須與** [[F-012#R1]]-[[F-012#R3]] **一致**，
  且**必須重用** `QuantityStepper` 元件，不得另行實作。

## 場景
### S2: 修改數量失敗 (驗證 R4)
Given 結帳頁 SKU-A 數量為 2; API 回 500
When 使用者點擊「+」
Then 行為如 [[F-012#S3]] (**不在此複製其步驟**)

```

注意寫法：R4 與 S2 只寫參照，不複製 F-012 的文字；F-012 修改時，F-020 自動「跟著」被影響。

變更事件：F-012 R3 改了

PM 決定：失敗時保留使用者輸入的新值並顯示「重試」按鈕，不再回復舊值。這改變既有行為 → 破壞性變更 (6.3)，F-012 由 1.2.0 → 2.0.0。

步驟 1：變更者做影響分析

```

$ dart run tool/spec_lint.dart --impact F-012
== 影響分析：F-012 ==
[depends_on] F-020 (docs/04-features/F-020-checkout.md)
[refs]       F-014 (docs/04-features/F-014-cart-badge.md)

```

(上方為示意輸出格式)

| 受影響者 | 關係 | 判斷 | 處置 |
|-------|--|------|--|
| F-020 | depends_on: F-012@1.2，且 R4、S2 直接參照 R3、S3 | 受影響 | 建立 Task，F-020 退回 draft，待 PM 確認 R4 的新行為；depends_on 改為 F-012@2.0 |
| F-014 | refs (角標只讀數量，不涉及失敗處理) | 不受影響 | 通知即可 |

步驟 2：Agent 被指派 F-020 的任務時

假設另一個 agent 此時拿到 T-0245：實作 F-020 R4。

| Preflight 檢查 | 結果 | 行動 |
|-----------------|---------------------------------------|------------------------|
| F-020 狀態 | 已被退回 draft | 停止 (G-2.6a：不得實作 draft) |
| depends_on 版本釘選 | 釘選 F-012@1.2，目前 F-012 為 2.0.0 (主版號不符) | 同上；在 STATUS.md 記錄 |

Agent 在 STATUS.md 寫入 (【HITL・事中澄清】)：

```

## 待釐清
- Q-09 (F-020)：F-012 已升至 2.0.0 (R3 改為「保留新值+重試」)。F-020#R4 要求與 F-012#R1-R3 一致。
  請確認 F-020#R4 的預期行為，並更新 `depends_on` 為 `F-012@2.0` 後將狀態改回 ready。
  影響任務：T-0245 (已暫停)。

```

版本釘選比對屬於 preflight 的人工／agent 檢查；9.2 的參考實作未包含此項，建議團隊自行擴充為 spec_lint 的規則 (目標：depends_on 釘選的主版號 ≠ 被依賴者目前主版號 → 報錯)。

步驟 3：人工處理後

【HITL・事前核准】PM 確認後更新 F-020 (depends_on: [F-012@2.0, ...]，R4 不變、S2 改參照新的 F-012#S3)，狀態回到 ready，T-0245 恢復。

這個範例要傳達的：1. 參照而非複製 (P2) 讓變更自動傳播到依賴者。2. 版本釘選讓 agent 能自己發現「我依賴的東西已經變了」。3. 狀態

機制（**draft** 不得實作）讓 agent 在規格不穩時自動停手。

10.5 多專案協作：App + API + 共用契約庫

情境

三個專案、三個 repo：

| 專案 | 角色 | 擁有 |
|--------------------|----------------------------------|-----------------|
| platform-contracts | Contract Owner | 所有跨專案契約、Epic、CR |
| shop-api | Provider（後端） | 契約的實作 |
| shop-app | Consumer（Flutter App，即 ShopLite） | App 端功能 |

目標 **E-2026-014「庫存感知購物車」**：App 要在商品剩餘不多時顯示「僅剩 N 件」提示（新功能 **shop-app:F-030**）。F-030 需要 API 提供 **low_stock**；目前 **C-STOCK-API@1.1** 沒有 **low_stock**。

步驟 1：Consumer agent 發現缺口，提 CR（不得自行改契約）

shop-app 的 Agent 在 preflight 讀到 F-030 的 **depends_on: C-STOCK-API@1.2**，但 **contracts.lock** 只有 **1.1.2**。依 AGENTS.md「跨專案規則」，它停止實作並在 **platform-contracts** 建立 CR：

```
---
id: CR-0007
type: cr
title: C-STOCK-API 新增 low_stock 欄位
status: draft
owner: contracts-team
requested_by: shop-app (agent, task T-0250)
target: platform-contracts:C-STOCK-API
---

## 動機與證據
shop-app F-030#R1 需要在剩餘 ≤ 5 件時顯示「僅剩 N 件」。目前 `GET /stock/{sku}` 只回 `available`。
由 App 端以 `available <= 5` 自行判斷會使門檻散落在各客戶端（日後調整需發版）。

## 建議變更
`StockResponse` 新增選用欄位 `low_stock: boolean`，門檻由後端決定。

```yaml
StockResponse:
 type: object
 required: [sku, available]
 properties:
 sku: { type: string }
 available: { type: integer, minimum: 0 }
 low_stock: { type: boolean, description: "後端判定的低庫存；缺省視為 false" } # 新增
```

## 相容性分析
- 新增**選用**欄位 → **相容 (MINOR)**：1.1.2 → **1.2.0**
- 舊版 App 忽略未知欄位（已確認 JSON 解析設定 `ignoreUnknownKeys`）

## 影響範圍
專案	影響
shop-api	實作 low_stock 計算與門檻設定
shop-app	F-030 使用；需升級 contracts.lock
shop-admin (其他 Consumer)	無影響（忽略新欄位）

## 建議發布順序
Expand：API 先上線 → Migrate：App 升級 → 無需 Contract 收斂（純新增）

## 決議
（待 contracts-team 填寫）
```

步驟 2：Contract Owner（人）審查並核准，發布 **stock-api/v1.2.0**

【HITL・事前核准】 CR 的決議權在 Contract Owner，提出 CR 的 agent 只能等待。

核准後契約庫更新 **contracts/stock/openapi.yaml**，更新 **compat-matrix.md**，並建立 Epic：

步驟 3：Epic：**platform-contracts/epics/E-2026-014-stock-aware-cart.md**

```

---
id: E-2026-014
type: epic
title: 庫存感知購物車
status: implementing
owner: pm-lin
---
## 目標與成功指標
- 低庫存商品在購物車顯示提示；超量加入的錯誤率下降 ≥ 50% (事件 `qty_409_rate`)

## 工作分解
#	子項目	專案	指向	依賴	狀態
1	契約 C-STOCK-API 1.2.0	platform-contracts	CR-0007	-	implemented
2	後端實作 low_stock	shop-api	shop-api:F-221	1	implementing
3	App 低庫存提示	shop-app	shop-app:F-030	1	implementing (可用 mock 先行)
4	消費者契約測試	platform-contracts	consumer-contracts/shop-app-stock.json	1	ready
5	整合驗證 (測試環境)	兩者	shop-app:T-0240	2, 3	draft

## 介面先行清單
- C-STOCK-API@1.2 (已發布, tag `stock-api/v1.2.0`)

## 發布順序與相容性
1. shop-api 發布 (支援 `low_stock`, 向下相容)
2. shop-app 發布 (使用 `low_stock`; 若欄位缺省則視為 false, 故即使 App 先上線也不會壞)
3. 無需收斂

## 風險與回滾
- API 回滾: 移除欄位對舊 App 無影響
- App 回滾: 不涉及資料遷移

```

步驟 4：各 repo 的 agent 各自工作

| Agent (repo) | 讀到什麼 | 做什麼 | 不做什麼 |
|--------------------------|---|--|---------------------------------------|
| shop-app Agent | AGENTS.md (跨專案規則) → STATUS → <code>contracts.lock</code> → F-030 → Epic 子項目 3 | 任務授權後把 <code>contracts.lock</code> 的 <code>stock-api</code> 升到 1.2.0；用契約庫產生 client；以 mock 回應開發 S1-S3；寫 consumer contract | 不修改契約；不等 API 上線才開工 (mock 可行) |
| shop-api Agent | AGENTS.md → F-221 → C-STOCK-API@1.2 | 實作 <code>low_stock</code> ，Provider CI 必須通過所有 consumer contract | 不更動 <code>required</code> 欄位；不改既有欄位型別 |
| platform-contracts Agent | Epic、CR | 維護 <code>compat-matrix.md</code> ；彙整子項目狀態 | 不替任一方寫業務邏輯 |

步驟 5：各 repo 的 STATUS.md (只記錄本 repo 的事)

```

# shop-app STATUS (節錄)
## 進行中
- T-0250 F-030: S1、S2 通過 (以 mock)，S3 待做
## 阻擋 / 外部依賴
- 無 (CR-0007 已核准, stock-api 1.2.0 已發布)
## 待釐清
- Q-1 (PI-001) 缺貨商品是否自動移除? 影響 F-013#R4 (SHOULD, 未實作; 與本 Epic 無關)

```

Epic 才是跨專案進度的唯一來源；shop-app 的 STATUS 不重複記錄 shop-api 的進度。

步驟 6：消費者驅動契約測試的意義

shop-app 把「我依賴 `available` (必填) 與 `low_stock` (選用, 缺省 false)」寫成 consumer contract。之後若有人要在 API 把 `available` 改名，Provider CI 立刻失敗，不需等到 App 在線上壞掉。

步驟 7：如果事情不順

| 狀況 | 規則 | 動作 |
|--------|---------|---|
| CR 被否決 | 回覆原因，結束 | shop-app Agent 將 F-030 的 <code>depends_on</code> 改為 1.1，並以「App 端判斷 <code>available <= 5</code> 」重寫 R1，狀態退回 <code>draft</code> 交人工審查 (【HITL・事前核准】)；不得直接實作 |
| 後端延誤 | 介面已先行 | App 以 mock 先完成；T-0240 整合驗證維持 |

| | | |
|------------|-------|---|
| | | draft，不阻擋其他子項目 |
| 發現契約有破壞性問題 | 走 6.3 | 新 CR，採 Expand → Migrate → Contract（7.2.5） |

這個範例要傳達的：1. 跨專案的衝突不是靠聊天解決，而是靠 **CR**、**Epic**、**契約版本**、**consumer contract** 四個可持久化的機制。2. Consumer agent 發現缺口時，**正確行為是停下並提 CR**，而不是「幫忙」修改別人的資產。3. 以介面先行加上 **Expand → Migrate → Contract**，讓各專案能獨立上線。

第 11 章 導入、治理與持續改進

11.1 角色與責任

| 角色 | 責任 | 不是 |
|----------------|--|----------------------|
| 規格擁有者（人） | 對規格內容負責；回答開放問題；核准 <code>draft → ready</code> | 不替 agent 寫程式 |
| 技術負責人 | 架構規則、Pattern Policy、Autonomy Policy、CI 閘門 | |
| Contract Owner | 共用契約的審查與發布、 <code>compat-matrix</code> | |
| Agent | 在權限內實作、驗證、記錄；遇不確定時停止 | 不得自行核准規格、擴張權限、修改他人資產 |
| 審查者（人） | 審查 PR 與規格變更；抽查追溯 | |

原則：核准權永遠在人（本準則所有 `[HITL]` 標記的總原則，見 1.4、附錄 D）。agent 可以撰寫草稿、提出問題、實作 `ready` 的規格，但 `draft → ready`、B 級動作、CR 決議皆由人決定。

11.2 規格變更流程

| |
|--|
| 提出變更（人或 agent 的 PR / CR） |
| ▼ |
| 判斷類型：澄清 / 相容 / 破壞（6.3） |
| ▼ |
| 影響分析（ <code>spec_lint --impact</code> ） |
| ▼ |
| 審查（規格擁有者；破壞性另需技術負責人） |
| ▼ |
| 合併 + 版本遞增 + 通知依賴者 |
| ▼ |
| 依賴者處置（建 Task、退回 <code>draft</code> 或僅通知） |

上述流程中的「審查」為 `[HITL・事前核准]` 節點：agent 不得審查或合併自己提出的規格變更。

11.3 規格健康度指標

| 指標 | 目標 | 說明 |
|----------------------------|----------|---|
| 規格導致的返工率 | 逐季下降 | 因規格缺漏／矛盾而退回的 PR 比例 |
| 待釐清平均等待時間 | ≤ 2 個工作日 | 人工回答速度，決定 agent 的產能 |
| MUST 需求測試覆蓋率 | 100% | G5 追溯檢查 |
| 「停止並回報」次數 | 不為零 | 為零代表 agent 在猜測而不是在提問（ <code>[HITL・事中澄清]</code> 的健康訊號） |
| <code>spec_lint</code> 通過率 | 100% | 主幹隨時為綠 |
| 規格過期比例 | < 5% | 規格與程式不一致的比例（抽查） |
| 跨專案 CR 平均處理時間 | ≤ 3 個工作日 | 協作瓶頸指標 |

11.4 失敗回饋迴路（P12）

每次 agent 的產出被退回或出錯，先分類，再修正「規格」，最後才修程式：

| 錯誤現象 | 最可能缺少的規格 | 修正位置 |
|------------|----------|----------------------|
| 做了不該做的功能 | 非目標、超出範圍 | PI、Feature「超出範圍」 |
| 改了不該改的檔案 | 任務範圍／禁區 | Task、AGENTS.md「禁區」 |
| 到處新增抽象層 | 模式政策 | Pattern Policy「禁止」 |
| 同一問題出現不同寫法 | 範例檔與沿用規則 | Pattern Policy 的範例路徑 |
| 錯誤路徑沒處理 | 邊界與錯誤表 | Feature |

| | | |
|------------------------|----------------|---------------------------|
| UI 漏了 loading/error 狀態 | 狀態矩陣 | UX 元件規格 |
| 自行猜測後出錯 | 「不確定時」與升級條件 | AGENTS.md、Autonomy Policy |
| 推翻既有決策 | ADR 的否決方案與重審條件 | ADR |
| 不同 repo 互相覆蓋 | 擁有權與 CR 規則 | AGENTS.md、7.2 |
| 狀態轉換漏做 | 轉換表與窮舉測試 | SM |

11.5 導入路線

| 階段 | 做什麼 | 成果 |
|----------|---|-------------------|
| 第 1 週 | AGENTS.md、STATUS.md、Feature 模板、Task 模板；CI 加入 G1-G2 | agent 開始不亂改、不重複犯錯 |
| 第 1 個月 | 補 Glossary & Invariants、ADR、Pattern Policy、OpenAPI；加入 spec_lint、架構測試（G3-G5） | 規格與驗證形成閉環 |
| 第 2-3 個月 | 狀態機與窮舉測試、UX token 與 Golden、Autonomy Policy；多功能並行導入 _index.md 擁有權矩陣 | 可安全並行 |
| 持續 | 多專案：契約庫、CR、Epic、consumer contract；含 agent 的產品：Agent / Tool Spec 與 eval | 可規模化、可審計 |

最重要的心法：不要試圖一次寫完所有規格。**從失敗開始**——每次 agent 做錯，就補上缺的那一份。規格會隨失敗進化，這才是 agentic 時代的規格工程。

附錄 A 檢查表

A.1 Feature Spec 審查檢查表

【HITL・事前核准】draft → ready 之前由人使用本表。

- ☐ Front-matter 完整；`status` 與內容相符；`depends_on` / `refs` 正確且帶版本（契約）
- ☐ 每條需求有強度詞（MUST / SHOULD / MAY），無模糊詞（3.4）
- ☐ 每條 MUST 至少一個場景；每個場景標明驗證對象
- ☐ 邊界與錯誤為表格，涵蓋離線、逾時、權限、庫存／資源不足等
- ☐ 有「超出範圍」
- ☐ 有 `verify`，路徑可執行
- ☐ 狀態類行為參照 SM；API 類行為參照 `operationId`；UI 參照 UX；皆未複製內容
- ☐ 沒有阻擋性開放問題；有的話，負責人與期限已填
- ☐ ≤ 300 行

A.2 Task 審查檢查表

- ☐ 目標指向 `ready` 的規格與具體需求 ID
- ☐ 「可修改範圍」為路徑 `glob`，與進行中任務無重疊
- ☐ 「禁止修改」列出共用層與契約
- ☐ 提供模仿範例的檔案路徑
- ☐ 完成條件全部可由指令判定
- ☐ 預估 ≤ 500 行；否則已拆分
- ☐ 有「不確定時」指引

A.3 PR 審查檢查表（人）

【HITL・事後審查】本表即為人工審查的檢查項目。

- ☐ 變更檔案皆在任務範圍內
- ☐ 需求 → 場景 → 測試對應完整（8.7 表格）
- ☐ 無刪除或放寬既有測試；無 `skip`
- ☐ 「自主決定」皆為可逆的小細節；涉及行為的疑義已走「待釐清」
- ☐ 圖、轉換表、Golden、`verify` 已同步
- ☐ 影響範圍已填且合理
- ☐ 無新增依賴（或已核准）；無硬編碼字串、色碼、金鑰

A.4 新專案啟動檢查表

- ☐ AGENTS.md ≤ 100 行且指令可執行
- ☐ STATUS.md 已建立
- ☐ PI（含非目標、已決定事項）
- ☐ Glossary & Invariants
- ☐ Architecture（含依賴規則）+ 對應架構測試
- ☐ Pattern Policy（含禁止清單與範例路徑）
- ☐ 至少一個 ADR
- ☐ Autonomy Policy
- ☐ CI：G1–G5
- ☐ （多專案）契約庫、`contracts.lock`、AGENTS.md 的跨專案段落

附錄 B 反模式清單

| 反模式 | 後果 | 改正 |
|---------------------|------------------------|-----------------------------|
| 規格寫成長篇敘述文 | agent 抓不到重點、context 爆量 | 結構化、編號、≤ 300 行 |
| 只有願景沒有非目標 | agent 過度實作 | 補非目標與超出範圍 |
| 同一資訊寫在多處 | 必然不一致 | 單一事實來源 + 參照 |
| 需求沒有驗證方式 | 無法判定完成 | 補場景與測試 |
| 狀態圖與程式脫節 | 圖腐爛、被忽略 | 轉換表驅動測試 |
| 圖片檔的架構圖 | 無法 diff、agent 讀不到 | Mermaid 文字化 |
| 讓 agent 「自己判斷」金額或權限 | 高風險錯誤 | 升級人工；寫成明確規則 |
| 任務單寫「修改相關檔案」 | 範圍失控、衝突 | 路徑 glob |
| 多個任務同時改共用檔案 | 合併衝突、誤刪 | 整合任務專責 |
| 契約變更一次切換 | 舊版 App 全壞 | Expand → Migrate → Contract |
| 直接修改他人 repo | 擁有權混亂 | CR |
| 為通過測試而修改規格或測試 | 失去驗證意義 | 停止並回報 |
| 把 STATUS.md 當流水帳 | 失去交接價值 | 只記狀態、阻擋、待釐清 |
| 從不更新 AGENTS.md | 指令過期、重複犯錯 | 列入 DoD；失敗回饋迴路 |

附錄 C 問題 → 去哪裡找

| 我想知道... | 去看 |
|------------------|--|
| 某個詞是什麼意思 | glossary-invariants.md |
| 某條規則永遠成立嗎 | INV 編號 |
| 這個模組可以依賴哪個模組 | architecture.md (ARCH) |
| 為什麼當初選這個方案 | ADR |
| 這裡可以用 Factory 嗎 | pattern-policy.md |
| 這個功能該怎麼運作 | Feature Spec (F-nnn) |
| 這個訂單能從 A 狀態到 B 嗎 | SM-*.md 轉換表 |
| API 的欄位與錯誤碼 | openapi.yaml (Contract) |
| 這個按鈕有哪些狀態 | UX-*.md 狀態矩陣 |
| 我可以自己做這件事嗎 | autonomy-policy.md |
| 現在進度與卡在哪 | STATUS.md ；跨專案看 Epic |
| 我改了這個會影響誰 | <code>spec_lint --impact <ID></code> |
| 我需要別的專案改契約 | 建立 CR (7.2.4) |
| 規格不夠清楚我該怎麼辦 | 8.4 決策樹：停止並回報 |

附錄 D Human in the Loop (HITL) 總覽索引

本附錄彙整全文所有 HITL 節點。標記定義見 1.4。

D.1 事前核准 (HITL · 事前核准)

| 位置 | 機制 | 決定者 | Agent 等待時做什麼 |
|------------------|--|------------------|------------------------|
| 0.5 | 修改本準則須經 PR 與技術負責人核准 | 技術負責人 | 提出 PR |
| 1.3 P10、4.15 | Autonomy Policy 的 B 級動作 (新增依賴、改 CI、遷移、大規模重構) | 技術負責人 | 提出計畫與理由 |
| 2.6 G-2.6c | draft → ready | 規格擁有者 | 只撰寫草稿與提問 |
| 4.8 | 清單外的設計模式 | 技術負責人 | 在 PR 說明要解決的具體問題 |
| 4.12 | 契約的破壞性變更 (走 CR) | Contract Owner | 提 CR |
| 4.16 | Runbook 中不可逆步驟 | 維運負責人 | 不執行，等待 |
| 4.17、5.5 | 產品內 agent：金額、刪除、對外發送須人工確認 | 終端使用者或營運人員 | 暫停並請求確認 |
| 4.19、7.2.1、7.2.4 | 跨專案變更請求 (CR) | Contract Owner | 在 STATUS 登記 CR 並停止相關工作 |
| 8.3 第 5 點、9.1 G9 | 新增依賴套件 | 技術負責人 | 在 PR 標記並等待 |
| 9.4 | 降低 eval 門檻或刪除案例 | 技術負責人 | 不得自行調整 |
| 10.3、10.4、10.5 | 範例中的套件衝突、F-020 規格確認、CR 決議 | 見各範例 | 見各範例 |
| 11.1、11.2 | 規格變更的審查 | 規格擁有者；破壞性另需技術負責人 | 不得審查自己的變更 |
| A.1 | Feature Spec 上線前審查 | 規格擁有者 | — |

D.2 事中澄清 (HITL · 事中澄清)

| 位置 | 機制 | 決定者 | Agent 等待時做什麼 |
|----------------|----------------------------|--------|--------------------|
| 1.3 P6、4.1 | 不確定時停下並記錄，不得猜測 | 規格擁有者 | 寫入 STATUS.md 「待釐清」 |
| 4.2、7.1.5 | 「待釐清」由人回答後寫回規格 | 規格擁有者 | 處理其他不受影響的任務 |
| 4.3、4.9 | 開放問題標明負責人與期限，阻擋性問題不得 ready | PM／負責人 | 不得自行決定 |
| 4.7 | ADR「重新檢視條件」成立時，agent 可提出質疑 | 技術負責人 | 提出，不自行推翻 |
| 8.3 第 6 點、8.4 | 認為規格有誤時走決策樹 | 規格擁有者 | 停止並回報 |
| 10.1、10.2、10.4 | 範例中的 Q-1、Q-04、Q-09 | PM | 暫停相關任務 |
| 11.3 | 「停止並回報」次數不為零，作為健康訊號 | 技術負責人 | — |

D.3 事後審查 (HITL · 事後審查)

| 位置 | 機制 | 決定者 | Agent 要提供什麼 |
|---------|-------------|-----|------------------------------|
| 8.7、A.3 | PR 由人審查 | 審查者 | 需求 → 場景 → 測試對應表、「自主決定」段、影響範圍 |
| 10.1 | 草稿規格交 PM 審核 | PM | 草稿與開放問題清單 |

D.4 例外升級 (HITL · 例外升級)

| 位置 | 機制 | 觸發條件 | Agent 的行為 |
|--------------|-------------|-----------|-----------|
| 1.3 P10、4.15 | 升級條件 | 見 8.5 | 停止並回報 |
| 2.6 G-2.6c | 遇到 draft 規格 | 規格非 ready | 停止並回報 |

| 2.0.0-2.0.9 | 遇到 critical 故障 | 故障 ready | 停止並回報 |
|-------------|-----------------------|--------------------------------------|----------------|
| 7.1.4 | 非機械性的合併衝突 | 衝突無法機械解決 | 停止並回報 |
| 8.2 | Preflight 任一項不通過 | 如基線為紅燈、範圍重疊 | 停止並回報 |
| 8.4、8.5 | 決策樹與升級條件 | 連續 3 次失敗、> 500 行、涉及金流／權限／個資／不可逆、安全問題 | 停止；回報現象與已嘗試的做法 |

D.5 常見誤用

| 誤用 | 後果 | 改正 |
|---------------|------------------------------|--------------------------|
| 口頭核准、未留紀錄 | 無法追溯，agent 下個 session 不知道已核准 | 核准寫進 PR、CR 決議或 STATUS.md |
| agent 審查自己的變更 | 失去獨立性 | 審查者必須是人 |
| 什麼都要人核准 | 人疲勞、橡皮圖章 | 低風險歸 A 級，事後審查即可 |
| 待釐清無人回答 | agent 產能停擺 | 負責人與期限；追蹤等待時間（11.3） |

D.6 已知缺口（建議下版補強）

以下是本準則目前尚未明訂的 HITL 細節，導入時請團隊自行決定：

- 1. **核准人的名單**：每個專案應在 **AGENTS.md** 列出誰是規格擁有者、技術負責人、Contract Owner。
- 2. **回覆時限**：除了 11.3 的指標外，各節點的回覆期限未逐一定義。
- 3. **核准紀錄格式**：目前只要求「可追溯」，未規定固定格式。
- 4. **緊急情況**：事故處理時，若需要繞過核准流程（例如緊急修復），本準則未定義替代流程與事後補核准的規則。

本準則為活文件。每次 agent 因規格不足而出錯，請回頭修正規格，並以 PR 更新本文件。